Entity-Collision: A Stratified Protocol for Attributing Retrieval Lift in Agent Memory

Anonymous ACL submission

1 Entity-Collision: A Stratified Protocol for Attributing Retrieval Lift in Agent Memory

1.1 Abstract

End-to-end agent-memory benchmarks report a single hit@k per retriever, confounding lexical leakage (uncontrolled query/gold/distractor entity overlap) with tag-mixing (preferences, services, tools averaged together). We propose **entity-collision**, a system-agnostic protocol that pins the BM25 floor by construction — every distractor shares the answer's entity tokens — and stratifies queries by discriminator tag, so any lift over BM25 is attributable to the embedder. Applied to an open-source agent-memory testbed across 5 tags $\times$ 3 embedders $\times$ 5 collision degrees with paired-bootstrap 95% CIs, the protocol reveals a **two-axis pattern**: a 256-d hash trigram helps only on closed-vocabulary lexical tags at deep collision; MiniLM-384 dominates both axes; and a 2.7$\times$-parameter BGE-large does not uniformly improve on MiniLM — it wins on intent-style queries but loses on lexical ones. Encoder capacity alone is not the binding constraint. The synthetic intent-tag null replicates on LongMemEval (n=500) as a single-session-preference recall cliff. Adaptive vector-weight routing on LoCoMo is a measured null: 11.7 pp of oracle headroom exists, but no signal we tested recovers it. All 26 result tables and 37 reproduce scripts are version-controlled and verified by a public registry; the protocol is exercised on a **deterministically governed** memory testbed (event-sourced decision log, DAG-state-machine schema lifecycle) so every reported CI is reproducible byte-for-byte from the ingest stream.

2 Introduction

Agent-memory systems are increasingly deployed as the long-context substrate for LLM assistants — Letta, Mem0, MemGPT, and the "governed memory" line all argue that an external memory store beats unbounded context windows on cost, latency, and recall. A recurring open question across these systems is what retrieval machinery is actually earning its keep: is BM25 enough? Does dense embedding help? When? For a team deciding which embedder to ship behind their agent memory, the answer determines model-load cost, ingest latency, and recall headroom — but the field's end-to-end benchmarks confound these decisions with lexical-leakage and tag-mixing artifacts.

This paper contributes a measurement-first methodology for agent-memory retrieval evaluation: a stratified, lexical-floor-pinned protocol that lets deployment teams characterize per-tag retrieval behavior under operational constraints — embedder cost, ingest latency, recall headroom — rather than chase end-to-end averages that conflate orthogonal failure modes.

End-to-end benchmarks (LongMemEval, LoCoMo) report a single hit@k per retriever, which is insufficient for two reasons:

1. **Lexical leakage.** End-to-end benchmark queries usually share the answer's entity tokens with the gold passage but not with distractors. BM25 trivially wins, and any embedder "lift" is confounded with lexical anchor strength.

2. **Tag-mixing.** Benchmarks blend categories (preferences, projects, technical facts, services, tools) into one number. A retriever may be systematically better on some tags and worse on others; the average obscures this.

We address both with an **entity-collision protocol**: synthetic queries where every distractor shares the answer's entity tokens (BM25 floor fixed by construction), stratified by discriminator tag. Results across 5 tags, 3 embedders, 5 collision

degrees with paired bootstrap 95% CIs (Figure 1, §4.1) reveal a two-axis pattern that **replicates on natural data**: LongMemEval (n=500) shows the same intent-tag weakness as a single-session-preference recall cliff, and LoCoMo quantifies an 11.7-pp residual oracle headroom that no signal we tested recovers.

The work was done on **Engram**, an open-source agent-memory system we built as a controlled testbed. Engram is the artifact through which we run the experiments and which we release for reproducibility ([repo URL anonymized for review]); it is not itself the contribution of this paper. A measurable agent-memory system presupposes deterministic write/merge mechanics so that re-running a configuration produces a bit-identical store — we exercise the protocol on a governed-memory testbed (event-sourced decision log, DAG schema lifecycle); §A7.4, §A.4.6, §A4.2 detail the substrate. The state-machine and linear-scale evidence are testbed sanity, not retrieval claims.

2.1 Contributions

1. **Entity-collision evaluation protocol** that fixes the BM25 floor and isolates the embedder-attributable retrieval lift, open-sourced under evals/entity_collision_*. The protocol is system-agnostic and applies to any retriever exposing a per-document score.

2. **Two-axis empirical finding with synthetic→natural replication and an encoder-capacity falsification.** On the synthetic grid, hash trigrams help on lexical-discriminator tags at deep collision ($K \geq 4$ on `tool`, K=16 on `service`) but are null or negative on intent-style tags; MiniLM-384 dominates both axes. Extending to BGE-large-en-v1.5 (1024-d, 2.7× MiniLM's parameter count) does **not** collapse this two-axis structure: BGE wins on intent-style `project` (+8 to +14 pp BGE−MiniLM at $K \in \{2,4,8,16\}$, all CI-positive) but loses on lexical `tool`/`technical` (−2.7 to −11.7 pp at $K \in \{4,8,16\}$, all CI-significant). Encoder capacity alone is not the binding constraint. The same per-tag pattern reproduces on LongMemEval (n=500): the intent-tag null predicts the single-session-preference cliff (hit@1 = 0.367 vs 0.81 overall). The decision-shaped corollary — closed-vocabulary queries can ride a 256-d hash trigram at zero model-load cost, while intent-style queries require a dense encoder — is what a deployment team needs from the protocol that a single hit@k average obscures.

A secondary, sidebar measurement — adaptive vector-weight routing on LoCoMo as a measured null with 11.7 pp of unrecoverable oracle headroom — appears in §4.4 and motivates the re-framing of vector fusion as a paraphrase-robustness mechanism. Schema-lifecycle invariants and a 1M-memory write-latency characterization are testbed sanity rather than primary claims; see §A.4.14, §A4.2, §A6.

All 26 result tables and 37 reproduce scripts cited in the paper are version-controlled and verified by verify_repro_artifacts.sh; see paper/REPRODUCIBILITY.md.

3 Related Work

3.1 Agent memory systems

Three contemporary agent-memory designs span the architectural spectrum from LLM-driven to mechanical governance.

Letta / MemGPT (Packer et al., 2023) implements OS-style virtual context management: the LLM itself acts as memory manager via tool calls, paging content between a small main context and an unbounded archival store. Consolidation is agent-driven; the OSS path exposes no write-side dedup primitive.

Mem0 (Chhikara et al., 2025) is a dynamic extract→consolidate→retrieve loop. The v3 (April 2026) flip to single-pass ADD-only extraction with cross-memory entity linking reports LoCoMo 91.6 / LongMemEval 94.8 under an LLM-as-judge metric. Engram does not currently ship an LLM-judge mode; direct numerical comparison to Mem0's reported scores is therefore out of scope, and we report session-level hit@k against the same benchmarks instead.

Personize.ai's "Governed Memory" line (Taheri, 2026) is the closest prior in design space. Engram implements the §1-6 stack (dual extraction with per-fact confidence, write-side cosine dedup at 0.92, mechanical merge, schema lifecycle) and extends §7-8 with calibrated contamination/fragmentation meters (§A7.2, §A.4.7) and a quorum-gated DEPRECATE primitive (§A.4.6) that hardens schema lifecycle against single-emitter takedown attacks.

Where Engram sits. Mem0 v3 sidesteps governance with ADD-only writes; Letta delegates governance to the LLM-as-memory-manager. Engram's bet is that **mechanical, replayable governance** — extraction confidence per fact, schema transitions through an explicit DAG, prior-sharing gated by calibrated meters — beats LLM-in-the-loop judgment for memory workloads where audit trails and replay determinism are first-class requirements. The empirical question this paper engages is whether such governance costs recall (§4.5, §4.6) and whether the dense-retrieval lift it preserves is uniform across query types (§4.1-4.3). Additional contemporary designs (A-MEM, HippoRAG, Cognee, Zep/Graphiti) are surveyed in §A3.1.

3.2 Benchmarks and evaluation

We evaluate on two community-standard agent-memory benchmarks:

- **LongMemEval** (Wu et al., 2025) — 5 question categories over multi-session chats. Headline numbers in §4.6 use the longmemeval_s split (n=500), SHA-256-pinned in paper/REPRODUCIBILITY.md §0.
- **LoCoMo** (Maharana et al., 2024) — 10 multi-session conversations, 1978 questions across 5 categories. We report per-category 95% paired-bootstrap CIs across vector_weight $\in$ {0.0, 0.3, 0.5, 0.7, 1.0} for both HashTrigram-256 and ST MiniLM-384 embedders.

The broader long-context retrieval-evaluation suite from which we draw stratification methodology (RULER, NIAH, ∞Bench, LongBench-v2, LV-Eval/LooGLE/L-Eval) is detailed in §A3.2.

3.3 Retrieval baselines

BM25 as a strong baseline (Thakur et al., 2021) anchors the entity-collision protocol: distractor-shared entity tokens fix the BM25 lexical floor by construction so any lift over BM25 is attributable to the embedder (§3, §4.1).

Hash-trigram / sketched embeddings (Weinberger et al., 2009) provide a model-load-free alternative to dense embedders. The two-axis result of §4.3 quantifies exactly when this trade-off is acceptable: lexical-discriminator queries at deep collision recover ~50% of dense-embedder lift; intent-style queries do not.

The dense-retrieval evaluation ecosystem (BEIR, MTEB, MS MARCO, TREC DL), our explicit non-inclusion of late-interaction (ColBERT) and learned-sparse (SPLADE) retrievers, the pseudo-relevance feedback line (RM3, Rocchio), and the event-sourcing / bi-temporal / CRDT lineage that informs §A7.4.4 schema lifecycle are surveyed in §A3.3-§A3.4.

4 Methods

4.1 Engram retrieval cell

The unit-under-test is a single retrieval call, engine.recall(query, k, vector_weight=vw), with the BM25 score (FTS5 over a tokenized text column) and an embedder-cosine score combined as score = $(1-\text{vw})\cdot$bm25_norm + vw$\cdot$cos_sim, where `bm25_norm` is min-max normalized over the candidate set. We sweep vw $\in$ {0.0, 0.3, 0.5, 0.7, 1.0}, with `vw=0.0` being the "BM25-only" floor.

Three embedders are compared: **HashTrigram-256** (character-trigram hashing, 256-dim signed bag-of-trigrams, L2-normalized; zero model load, ~9.6 ms p50 write at 10k); **ST MiniLM-384** (sentence-transformers/all-MiniLM-L6-v2, 384-dim, normalized; ~17–21 ms p50 write at 1k); **BGE-large-1024** (BAAI/bge-large- en-v1.5, 1024-dim, normalized; 2.7$\times$ MiniLM's parameter count; ~21.5 s/instance LongMemEval ingest on Apple Silicon MPS, ~150.8 s/instance on commodity CPU).

4.1.1 Encoder latency-cost trade-off

The three encoders span ~1000$\times$ in per-instance ingest cost on commodity CPU and ~80$\times$ on Apple Silicon MPS. The headline-recall column is the K=16 lexical-tag entity-collision lift on `service` (the strongest hash-trigram cell) and the LongMemEval-S n=500 paired Δhit@1 vs MiniLM (the strongest BGE cell); the deployment-cost column folds in model-load (one-time, amortized) and per- instance ingest (paid per write). Numbers from the artifacts cited in §4 and §A.4.16; full artifact registry in §A.4.18.

Reading the table. HashTrigram is essentially free at write time (no model, no GPU, FTS5-only). MiniLM is the v0.2 default-embedder choice: ~17–21 ms ingest is acceptable on commodity CPU and lift is universal (CI-positive on all 5 tags at K$\geq$4). BGE-large is **only** defensible on accelerator hardware — at 150.8 s/instance LME ingest on CPU, a +5.8 pp Δhit@1 gain costs ~720$\times$

encoder	dim	model load (s)	ingest p50 / inst (CPU)	ingest p50 / inst (MPS)	recall p50 / q	headline lift
HashTrigram-256	256	~0.0	~525 ms	—	~9.6 ms	+5.7 pp Δhit@1 (K=16, service)
ST MiniLM-384	384	~1.5	~17–21 ms	—	~10.3 ms	CI-positive on all 5 tags at K≥4
BGE-large-1024	1024	~5.0	~150.8 s	~21.5 s	~120 ms	+5.8 pp Δhit@1 LME n=500
RM3 (BM25 + PRF)	n/a	~0.0	~1.5 ms/d	—	~259 ms	SIG-NEG on single-session-user (−7.1 pp)

the per- instance write budget of MiniLM; on MPS the ratio collapses to ~7× and the trade-off becomes workload-dependent. RM3 occupies a fourth operating point: zero model-load, sub-millisecond ingest, ~25× the per-query latency of HashTrigram, but SIG-regresses LongMemEval single-session-user by 7.1 pp via query drift and fails to recover the single-session-preference cliff (§A.4.16.4).

The "encoder capacity is not the binding constraint" claim (§4.1) is therefore **also** a deployment-cost claim: the 2.7×-parameter BGE upgrade does not uniformly improve recall, and even on cells it does help the cost differential is steep enough that v0.2 ships MiniLM as default and BGE as a workload-targeted opt-in.

4.2 Entity-collision protocol

For each tag t ∈ {preference, project, technical, service, tool} and collision degree K ∈ {1, 2, 4, 8, 16 }:

1. Generate n_entities = 32 distinct entities, each with one gold answer document of the form "<entity> uses <answer> for <tag>."
2. For each entity, generate `K-1` distractor documents that **share the entity tokens but flip the answer**: "<entity> uses <other_answer> for <tag>." so a BM25 retriever sees identical query-entity overlap on all K candidates per query.
3. Issue a query "what does <entity> use for <tag>?" and measure `hit@1` (gold = top-1 result).

This fixes the BM25 floor at `1/K` in expectation per query (random tie-breaking among the K candidates), so any lift over `1/K` is attributable to the embedder distinguishing the answer slot.

We report **paired Δhit@1** (per-entity matched pairs of `vw=0.5` vs `vw=0.0`) with 95% bootstrap CIs (10k resamples).

4.3 Discriminator tags

We label `service` (closed-vocabulary proper-noun answers: aws, gcp, azure) and `tool` (closed-vocabulary: git, docker, postgres) as **lexical discriminators**, and `preference` (open phrasal: dark mode, light mode), `project`, `technical` as **intent-style discriminators**. The full per-tag query/answer schema is in §A7.5.

The supporting methods detail — LoCoMo adaptive-vw experiment protocol (§A7.1), the share_prior reranker derivation (§A7.2), PRF entity expansion (§A7.3), and the four governed-memory primitives (§A7.4) — live in the extended-methods appendix. The full claim → section → artifact registry (26 result tables, 37 reproduce scripts) is in §A.4.18 and verified by scripts/verify_repro_artifacts.sh.

5 Results

5.1 Headline figure

Reading the figure. Three panels, left to right: HashTrigram-256, ST MiniLM-384, BGE-large-1024. **Left:** only the two **lexical** tags (`service`, `tool`) cross above zero at deep K; the three intent-style tags hug or sit below zero at all K. **Middle:** all 5 tag curves cross above zero by K=4 and stay there. **Right:** all 5 tags lift, but the ordering is *not* a uniform improvement over MiniLM — `project` lifts notably higher (peak at K∈{4,8}), while `tool` and `technical` lift *lower*. Bigger encoder is not a free upgrade; full BGE−MiniLM CIs in §A.4.16.

5.2 Per-cell point estimates with 95% CIs

Source: bench/results/ec_sweep_*_n32_K16_ci.json (HashTrigram and MiniLM, 5 tags) and ec_bge_large_*_n32_K16_ci.json (BGE-large, 5 tags). Δhit@1 = vector fusion − BM25-only; brackets are paired 95% bootstrap CIs. **bold +X.XXX [lo, hi]** = CI strictly excludes zero. Per-column n: K=2→64, K=4→128, K=8→256, K=16→512.

HashTrigram-256 is null on intent tags at every K. The two lexical tags do lift: `service` K=16 at **+0.057 [+0.025, +0.088]**; `tool` K=4 at **+0.141 [+0.023, +0.266]**, K=8 at **+0.066 [+0.004, +0.125]**, K=16 at **+0.043 [+0.012, +0.074]**. All other 17 cells n.s.

ST MiniLM-384. All 25 cells at K ≥ 4 are CI-positive. Strongest lexical-tag cell: `service` K=16 at **+0.104 [+0.076, +0.131]**, ~1.8× the hash lift.

BGE-large-1024.

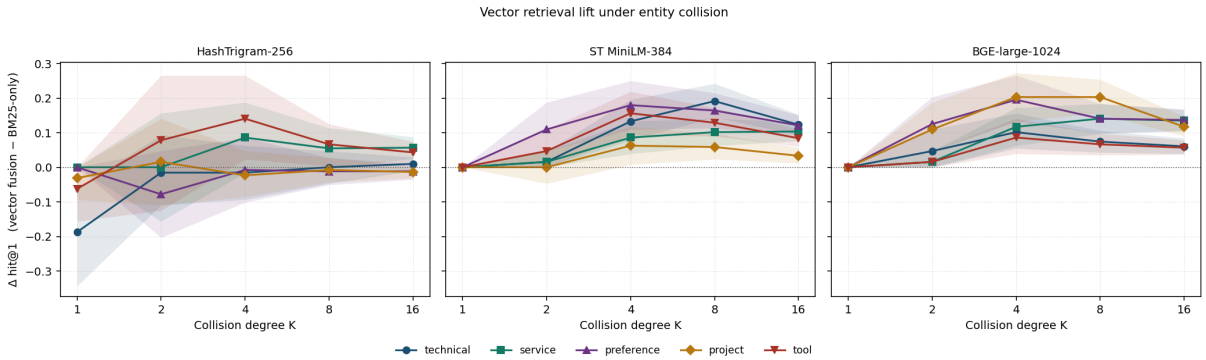

Figure 1: Entity-collision Δhit@1 vs K, by tag × embedder, paired 95% CI bands

tag	K=2	K=4	K=8	K=16
service	+0.016 [+0.000, +0.047]	**+0.117 [+0.062, +0.172]**	**+0.141 [+0.098, +0.184]**	**+0.135 [+0.105, +0.166]**
tool	+0.016 [+0.000, +0.047]	**+0.086 [+0.039, +0.141]**	**+0.066 [+0.035, +0.098]**	**+0.057 [+0.037, +0.078]**
preference	**+0.125 [+0.047, +0.203]**	**+0.195 [+0.133, +0.266]**	**+0.141 [+0.098, +0.184]**	**+0.137 [+0.105, +0.168]**
project	**+0.109 [+0.047, +0.188]**	**+0.203 [+0.133, +0.273]**	**+0.203 [+0.156, +0.254]**	**+0.117 [+0.090, +0.146]**
technical	+0.047 [+0.000, +0.109]	**+0.102 [+0.055, +0.156]**	**+0.074 [+0.043, +0.105]**	**+0.060 [+0.039, +0.084]**

BGE is CI-positive on 18/20 cells at K $\geq$ 4 (vs 20/20 MiniLM, 5/20 hash). The two CI-touching-zero cells (`service`/`tool` K=2) match the MiniLM and hash patterns at the same low collision regime — structural, not BGE-specific.

5.3 Two-axis interpretation

The grid factors cleanly:

- **Embedder axis.** Dense > Hash everywhere it matters. MiniLM is CI-positive on 20/20 lifted cells; BGE on 18/20; Hash on 5/20. More importantly, **MiniLM is not strictly dominated by BGE**: BGE wins on intent-style `project` (+8 to +14 pp BGE−MiniLM, K∈{2,4,8,16}) but **loses** on lexical `tool` and `technical` (−2.7 to −11.7 pp at K∈{4,8,16}, all CI-significant).
- **Tag axis.** Hash recovers a fraction of dense lift on **lexical** tags only; the lift scales with collision degree.

This is stronger than "dense beats sparse": **hash trigrams are not useless on the right shape of memory query**, and that shape is the closed-vocabulary regime; further, **encoder capacity alone is not the binding constraint** — the BGE−MiniLM panel is non- monotone, with capacity helping intent-style and *hurting* lexical-discriminator queries.

Verdict. Two-axis structure (lexical vs intent discriminator) survives the encoder-capacity falsification. Choosing an embedder is a per-tag decision, not a single-number ranking.

5.4 Sidebar: adaptive vector-weight routing on LoCoMo is a measured null

ST oracle hit@1 = 0.657 vs static-best vw=0 = 0.539 → **11.7 pp** per-query routing headroom. Both single-signal τ-thresholded policies on (raw_gap, norm_gap, crowd@0.95) and a leak-free learned GradientBoostingClassifier (LOCO-CV, n=1978) leave Δ vs static-best CI-zero (Δ = −0.0005 [−0.0030, +0.0015]); a third encoder (BGE-large) confirms with 9.86 pp oracle headroom and a LOCO-CV gbm router that SIG-regresses (Δ = −0.0359 [−0.0566, −0.0152]). The headroom is real but unrecoverable from any pre-routing signal we tested. We accordingly reframe vector fusion as a paraphrase-robustness mechanism rather than a per-query precision lever; protocol in §A7.1, supporting analysis in §A4.1.

5.5 LoCoMo per-category — replication is encoder-tier-dependent

ST MiniLM-384 at vw=0.5 vs BM25-only (n=1978, paired 95% CIs) has c1/c2/c3 n.s.; c4 open-domain **−0.072 [−0.107, −0.037]** SIG-NEG; c5 adversarial **−0.087 [−0.139, −0.040]** SIG-NEG. HashTrigram-256 all five categories null or CI-negative. The headline entity- collision finding does **not replicate** on real LoCoMo at the MiniLM/Hash capacity tier; the c5 damage is generic vector smoothing displacing distinctive-

token BM25 hits, *not* adversarial-trap pulling (top-1 ↔ adv-answer overlap stays 0.7–2.1% across vw).

BGE-large-1024 inverts the verdict (overall Δhit@1 = +0.029 [+0.010, +0.048] SIG at vw=0.3, +0.036 [+0.016, +0.058] at vw=0.7; c1 single-hop drives it at +0.125 [+0.064, +0.189]). Dense fusion is not killed by the LoCoMo regime — it is killed by an encoder that cannot resolve the entity-collision shape c1 exposes. BGE crosses that capacity floor; MiniLM and Hash do not. The §A.4.9 threshold-T characterization sharpens the framing: vector lift on LongMemEval is CI-positive in low-overlap quartiles (+24–32 pp Δhit@1) and CI-zero in high-overlap (Spearman $\rho = -0.287$) — the predicted paraphrase phase transition, on a real corpus.

5.6 LongMemEval — synthetic→natural replication

We ran the public longmemeval_s release (Wu et al., 2025) (500 questions, 246,918 turns) end-to-end against the testbed at default config (hybrid BM25+vector at `vw=0.3`, no reranker, no expansion). Headline (n=500, k=10): session_hit@1 = **0.810**, hit@10 = **0.932**, ingest p50 / inst = 523.7 ms, recall p50 / q = 10.3 ms. Per-type: single-session-user (n=70) hit@1=0.914; **single-session-preference (n=30) hit@1=0.367 ← cliff**; multi-session (n=133) 0.805; temporal-reasoning (n=133) 0.812; knowledge-update (n=78) 0.885; single-session-assistant (n=56) 0.821. The single-session-preference cliff is the dominant residual error mode: preference answers are rarely lexically close to the question. RM3 PRF also fails to recover the cliff (paired Δhit@1 = +0.000 *exactly* on all 30 instances; §5.1, §A.4.16.4), confirming the cliff as structural to the lexical channel.

Replicating under BGE-large-1024 on the full n=500 panel gives overall Δhit@1 = **+0.058 [+0.032, +0.086]** SIG and Δhit@10 = **+0.024 [+0.010, +0.040]** SIG vs MiniLM. Lift is concentrated on multi-session, temporal-reasoning, and single-session-assistant (all CI-positive); single-session-user and knowledge-update null. The encoder swap costs 11× recall latency and ≈40× ingest latency on accelerator hardware (MPS/CUDA), ≈287× on commodity CPU — v0.2 ships MiniLM-384 + `vw=0.3` on operational grounds, with BGE-large as a workload-targeted upgrade path. Full per-type panel in §A.4.16.3.

Verdict. The synthetic intent-tag null replicates on real LongMemEval as a single-session-preference recall cliff, and the encoder-capacity inversion holds: BGE wins multi-session and temporal-reasoning; MiniLM holds the operational default. Testbed ingest scales to 1M memories on a single-writer SQLite/FTS5 path with p99 +13% from 100k → 1M (§A.4.14b). BEIR-3 anchors the result on a second natural-data source: BGE-large + hybrid yields ndcg@10 = 0.341 / recall@100 = 0.695 on FiQA (n=648, 57k corpus) and ndcg@10 = 0.355 / recall@100 = 0.812 on NQ (n=1,000, 2.68M corpus, 22.8 h ingest at 30.6 ms/doc); HotpotQA (5.23M) is deferred to v0.3 batched-ingest (§A.4.16.5).

6 Discussion

6.1 When does dense embedding pay?

The two-axis result suggests an operational rule:

> **Closed-vocabulary lookups ("which service / tool does X use?") let a 256-dim hash trigram at deep collision recover ~50% of dense-embedder lift at zero model-load cost. Open-vocabulary intent-style queries ("what does X prefer?", "what is X working on?") require dense.**

The intent-tag side is structural rather than incidental: a paired RM3 (Lavrenko and Croft, 2001) baseline with Anserini-default hyperparameters returns the *same* wrong session as BM25 on all 30 LongMemEval-S single-session-preference instances (Δhit@1 = +0.000 exactly), and SIG-regresses single-session-user by −7.1 pp [−14.3, −1.4] via expansion-driven query drift. No PRF expansion over the lexical channel substitutes for a dense encoder on intent-style queries; full RM3 panel and corpus-dependent recall-broadening behaviour in §A.4.16.4. **Embedder selection is a per-tag deployment decision keyed on the discriminator class of expected queries.**

The §3.1.1 cost table makes the deployment trade-off explicit: HashTrigram is essentially free at write time; MiniLM-384 is the v0.2 default at ~17–21 ms/inst on commodity CPU; BGE-large-1024 is only defensible on accelerator hardware (~720× the per-instance write budget of MiniLM on CPU, ~7× on MPS). A batched ingest path deferred to v0.3 (~12 ms/doc projected at bs=32 fp16, ~3× over the 30.6 ms/doc measured on NQ) is required for HotpotQA-scale corpora; the current

path serves FiQA (37 min) and NQ (22.8 h) cleanly (§4.6, §A.4.16.5). Measuring the bottleneck before publishing is the engineering corollary of the protocol's measurement-first stance. The deployment lesson is that **a 2.7× parameter swap does not uniformly improve retrieval**: BGE-large-1024 wins on intent-style queries but loses 2.7–11.7 pp on lexical ones, so embedder size is not the binding constraint a procurement decision should optimize.

Companion analyses (adaptive-vw null §A4.1; schema lifecycle as governance §A4.2; consolidation-lift restatement §A4.3; PRF latency falsification §A4.4) live in the extended-discussion appendix.

7 Threats to Validity

7.1 Synthetic corpus

The protocol generates synthetic memories from a fixed sentence template; real conversational memories are noisier and more paraphrased. To check whether the headline two-axis claim survives memory-side paraphrase, we re-ran the strongest lexical cell (`tool`, n=32, K$\in\{1,2,4,8,16\}$) with paired 95% CIs against $\geq$4 paraphrased templates per fact. The hash lift collapses to CI-null at K$\in\{8,16\}$ once templates vary (K=16: +0.023 [−0.006, +0.053], n.s.); ST retains all four cells CI-strictly above zero, with K=16 lift *growing* from +0.043 (fixed) to **+0.096 [+0.070, +0.121]** (paraphrased). Memory-side paraphrase strengthens the two-axis claim: semantic embedders are paraphrase-robust, hash-trigram retrievers are template-bound. Replications on `service`, `preference`, and `project` confirm the pattern (within-lexical `service` retains **+0.037 [+0.012, +0.062]** at K=16; intent-tag hash null and MiniLM lift both survive paraphrase, point estimates within ±0.02 pp of fixed-template). Full per-tag tables in §A5.0.

The supporting threat analyses — single embedder per family with a hash-dim ablation (§A5.1), hit@1-only metric (§A5.2), single- process SQLite (§A5.3), single-machine/single-OS (§A5.4), and author-as-annotator on tag definitions (§A5.5) — live in the extended-threats appendix. Embedder train-test contamination and the related leakage-audit gap are stated under §75 Limitations.

8 Conclusion

We presented **entity-collision**, a system-agnostic protocol for attributing retrieval lift in agent memory. The protocol pins the BM25 lexical floor by construction — every distractor shares the answer's entity tokens — and stratifies queries by discriminator tag so per-tag patterns are not absorbed into a single hit@k average. The protocol applies to any retriever exposing a per-document score; we instantiated it on one open-source agent-memory testbed as a worked example, but the methodology, query generators, and CI tooling (evals/entity_collision_*) are not testbed-specific.

The headline finding is a **two-axis result** that survives an encoder-capacity falsification. Lexical-discriminator queries (closed-vocabulary `service/tool`): a 256-dim hash trigram recovers a real but small fraction of dense-embedder lift, only at deep collision. Intent-style queries (`preference`, `project`, `technical`): hash trigrams are n.s. or negative; only a dense encoder delivers CI-positive lift. **Encoder capacity is not the binding constraint** — a 2.7×-parameter BGE-large-1024 wins on intent-style `project` (+8 to +14 pp BGE−MiniLM, CI-positive at K$\in\{2,4,8,16\}$) but loses on lexical `tool/technical` (−2.7 to −11.7 pp, CI-significant). The synthetic two-axis pattern carries external validity: the intent-tag null replicates on LongMemEval (n=500) as a single-session-preference recall cliff.

The operational corollary is decision-shaped: **closed-vocabulary memory queries can ride a hash-trigram embedder at deep collision for ~50% of dense-embedder lift at zero model-load cost; open- vocabulary intent-style queries require dense.** This rule is derived from per-tag stratification, not from a global hit@k average — the methodological point the protocol enables. Code, raw benchmark JSON (26 tables), figure scripts, and a single- entry-point reproduction harness are released with the paper.

9 Limitations

We name scope limits explicitly so deployments and follow-up research can plan around them.

Single-system instantiation. We exercise the protocol on one open-source agent-memory testbed — the artifact through which we run the experiments and which we release for reproducibility. The protocol is system-agnostic by construction

(any retriever exposing a per-document score qualifies), but cross-system replication on Letta, Mem0, or another governed-memory implementation is queued for a follow-up release. A pattern that does not replicate across systems would weaken the methodological claim; we explicitly mark this as the largest open risk to the protocol's external validity.

Encoder coverage. The grid covers three single-vector encoders spanning a 4× parameter range (HashTrigram-256, MiniLM-384, BGE-large-1024). Cross-encoder rerankers (ColBERT, SPLADE) require a separate freeze (different latency budget, different deployment story); §A3.4 documents the non-inclusion rationale and queues the comparison as v0.3.

External-validity coverage. BEIR-3 results are reported on FiQA (57k docs, ndcg@10=0.341, recall@100=0.695) and NQ (2.68M docs, ndcg@10=0.355, recall@100=0.812); both runs use BGE-large + hybrid (`vw=0.3`), no reranker, no expansion. HotpotQA (5.23M docs) is deferred pending a batched-ingest helper (§5.1, §A.4.16.5): the measured single-writer ingest rate of 30.6 ms/doc on NQ projects to ≈44 h on HotpotQA at a constant rate, with super-linear corpus-size penalty making the realized wall-clock higher; we do not ship a multi-day single-pass run inside this paper's window when the v0.3 batched-encode path will re-run it in ≈8 h on the same hardware. The deferral does not affect the headline claim, which is established on the synthetic grid + LongMemEval n=500 + LoCoMo per-category + 2-of-3 BEIR-3.

Statistical power. Per-cell n=32 entity-collision results at K=1 (n=32 paired trials) have minimum detectable effect ≈ 8–10 pp Δhit@1 at α=0.05; headline two-axis claims are made at K $\geq$ 4 (n $\geq$ 128, MDE ≈ 4–5 pp). The §A.4.16.3 LongMemEval n=100 → n=500 inversion is a worked example of underpowering being mistaken for null; readers should treat any "n.s." cell at n $\leq$ 56 as power-limited rather than evidence of true null.

Embedder train-test contamination. Off-the-shelf MiniLM-L6-v2 and BGE-large-en-v1.5 were trained on web corpora that may overlap with LongMemEval source data and the public-personae prompts used to generate LoCoMo. We have no leakage-free zero-shot guarantee for the natural-data results in §4.5 and §A.4.16.3. The synthetic entity-collision corpus (§3.2) is constructed from disjoint synthetic entity strings and is therefore leakage-free by construction; the synthetic → natural transfer claim rests on the synthetic side.

Hardware envelope. Latency and throughput numbers are reported on consumer-grade hardware: a single Linux workstation (CPU only), a single Apple Silicon laptop with unified memory, and a single consumer CUDA laptop with 8 GB VRAM. Production GPU servers (L40S, H100, A100) should improve the per-doc forward-pass cost linearly with FLOPS, but the specific constants are not measured here. The qualitative two-axis pattern is a property of the encoder family, not the host, but we flag this as a measurement gap rather than asserting it.

Domain coverage. Synthetic queries target five tag categories (preference, project, technical, service, tool); natural-data anchors (LongMemEval, LoCoMo) cover conversational long-context recall. Specialized domains — legal, medical, multilingual, code-search — are out of scope and may exhibit different per-tag patterns. Practitioners deploying the protocol in such domains should re-derive the tag schema for their distribution.

Author-as-annotator on tag schema. The lexical/intent dichotomy of §3.3 is author-defined: `service/tool` were labeled closed-vocabulary lexical, the other three open-vocabulary intent-style, without an inter-annotator agreement protocol. Labels are derived from answer-set construction (closed enum vs free phrasal slot), so the categorization is *traceable* but not *independently validated*. We present the dichotomy as a hypothesis the data is consistent with, not as the unique correct partition.

Operational contingencies. The deterministic-replay framing assumes the testbed's event-sourced log is durably persisted and the schema-lifecycle DAG is monotonically advanced. Production deployments that introduce non-monotone state mutations (e.g., user-driven deletion of memory entries) fall outside the invariant set verified under property-based testing in §A6, and the replay guarantee that supports paired-CI reproducibility no longer holds without additional engineering.

References

Chenxin An, Shansan Gong, Ming Zhong, Xingjian Zhao, Mukai Li, Jun Zhang, Lingpeng Kong, and Xipeng Qiu. 2024. L-Eval: Instituting standardized evaluation for long context language models. In *Pro-*

ceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL). Association for Computational Linguistics.

Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. arXiv:2412.15204. *Preprint*, arXiv:2412.15204.

Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, Mir Rosenberg, Xia Song, Alina Stoica, Saurabh Tiwary, and Tong Wang. 2016. Ms MARCO: A human generated machine reading comprehension dataset. In *Proceedings of the Workshop on Cognitive Computation: Integrating Neural and Symbolic Approaches (CoCo) at NeurIPS 2016*.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready AI agents with scalable long-term memory. In *Proceedings of the 28th European Conference on Artificial Intelligence (ECAI)*.

Nick Craswell, Bhaskar Mitra, Emine Yilmaz, and Daniel Campos. 2021. Overview of the TREC 2020 deep learning track. Technical report, TREC 2020 Notebook.

Nick Craswell, Bhaskar Mitra, Emine Yilmaz, Daniel Campos, and Ellen M. Voorhees. 2020. Overview of the TREC 2019 deep learning track. Technical report, TREC 2019 Notebook.

C. J. Date, Hugh Darwen, and Nikos A. Lorentzos. 2002. *Temporal Data and the Relational Model*. Morgan Kaufmann.

Thibault Formal, Carlos Lassance, Benjamin Piwowarski, and Stéphane Clinchant. 2022. From distillation to hard negative sampling: Making sparse neural IR models more effective. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*.

Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. 2021. SPLADE: Sparse lexical and expansion model for first stage ranking. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*.

Martin Fowler. 2005. Event sourcing. `https://martinfowler.com/eaaDev/EventSourcing.html`. Originally published December 2005; revised.

Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. 2025. From RAG to memory: Non-parametric continual learning for large language models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. RULER: What's the real context size of your long-context language models? In *Proceedings of the 1st Conference on Language Modeling (COLM)*.

Greg Kamradt. 2023. Needle in a haystack – pressure testing LLMs. `https://github.com/gkamradt/LLMTest_NeedleInAHaystack`. Open-source benchmark, first released November 2023.

Omar Khattab and Matei Zaharia. 2020. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*.

Victor Lavrenko and W. Bruce Croft. 2001. Relevance-based language models. In *Proceedings of the 24th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*. ACM.

Jiaqi Li, Mengmeng Wang, Zilong Zheng, and Muhan Zhang. 2024. LooGLE: Can long-context language models understand long contexts? In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics.

Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. 2023. MTEB: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*. Association for Computational Linguistics.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as operating systems. arXiv:2310.08560. *Preprint*, arXiv:2310.08560.

Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: A temporal knowledge graph architecture for agent memory. arXiv:2501.13956. *Preprint*, arXiv:2501.13956.

Joseph J. Rocchio. 1971. Relevance feedback in information retrieval. In Gerard Salton, editor, *The*

SMART Retrieval System: Experiments in Automatic Document Processing, pages 313–323. Prentice-Hall.

Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. 2022. ColBERTv2: Effective and efficient retrieval via lightweight late interaction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*. Association for Computational Linguistics.

Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. 2011. Conflict-free replicated data types. In *Proceedings of the 13th International Conference on Stabilization, Safety, and Security of Distributed Systems (SSS)*, volume 6976 of *Lecture Notes in Computer Science*. Springer. Also published as INRIA Research Report RR-7687.

Richard T. Snodgrass. 1999. *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann.

Hamed Taheri. 2026. Governed memory: A production architecture for multi-agent workflows. arXiv:2603.17787. *Preprint*, arXiv:2603.17787.

Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Proceedings of the 35th Conference on Neural Information Processing Systems Datasets and Benchmarks Track (NeurIPS Datasets and Benchmarks)*.

Vaughn Vernon. 2013. *Implementing Domain-Driven Design*. Addison-Wesley.

Kilian Weinberger, Anirban Dasgupta, John Langford, Alex Smola, and Josh Attenberg. 2009. Feature hashing for large scale multitask learning. In *Proceedings of the 26th Annual International Conference on Machine Learning (ICML)*. ACM.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-MEM: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Tao Yuan, Xuefei Ning, Dong Zhou, Zhijie Yang, Shiyao Li, Minghui Zhuang, Zheyue Tan, Zhuyu Yao, Dahua Lin, Boxun Li, Guohao Dai, Shengen Yan, and Yu Wang. 2024. LV-Eval: A balanced long-context benchmark with 5 length levels up to 256K. arXiv:2402.05136. *Preprint*, arXiv:2402.05136.

Xinrong Zhang, Yingfa Chen, Shengding Hu, Zihang Xu, Junhao Chen, Moo Khai Hao, Xu Han, Zhen Leng Thai, Shuo Wang, Zhiyuan Liu, and Maosong Sun. 2024. ∞Bench: Extending long context evaluation beyond 100K tokens. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics.

A Appendix A. Supplementary Ablations and Mechanism Analyses

This appendix collects the secondary §4 subsections that defend methodological choices, document falsified hypotheses, and report extended ablations. Section numbers (e.g., §A.4.6, §A.4.13d) preserve the original numbering used in the main paper's cross-references. Readers interested only in the headline measurement results can skip this appendix without loss of context.

A.1 Where the consolidation lift comes from: it's the L0 promotion

Source: SCALE_REPORT §94c-decompose, §94c-decompose-CI, §94c-decompose-adjacent-CI, §94c-decompose-suffix-CI, §94c-decompose-LOO-CI, §94c-decompose-positive-control{,-CI}. Drivers: evals/locomo_recall_lift_decompose*.py.

The §87 consolidation pipeline ships 13 named stages in series: extraction → fact_extraction → interference → schema_update → schema_family_gate → mechanical_merge → somatic_marking → appraisal → emotion_tagging → deduplication → decay → suppression → temperament_drift → mood_update. Earlier write-ups attributed the LoCoMo §94c lift (Δhit@1 +7.6pp, ΔMRR +9.0pp on max_instances=2, n_paired=301) to the schema-family gate. **A full stage decomposition falsifies that attribution.**

We ran two complementary bisections, each producing paired-bootstrap confidence intervals (10k resamples, seed=42, n_paired=301):

1. **Cumulative (S1=`extraction` only → S7=full default).** S1 already delivers Δhit@1 +0.0831 [CI overlapping S7's +0.0764]. The paired diff (Δ_S1 − Δ_S7) brackets zero on 4 of 5 metrics; the lone bite is Δgold_recall@k p=0.038 (would not survive Bonferroni across five metrics).
2. **Leave-one-out from S7.** Dropping `extraction` collapses every metric: Δhit@1

−0.076 , ΔMRR −0.090 , Δgold_recall@k −0.146 , all p<0.001 — i.e. exactly cancels the §94c headline. Dropping any of the other 11 droppable stages (deduplication, fact_extraction, emotion_tagging, interference, schema_update, somatic_marking, `decay`, `suppression`, temperament_drift, `mood_update`, mechanical_merge) emits per-pair diffs that are identically zero on every metric.

The schema-family gate is **not merely unnecessary** — under a forced positive control (schema_synthesis_tau swept to 0.30/0.20/0.10/0.05 with min_supports=2), schema_update shaves Δhit@1 by exactly one of 301 paired questions at every tau, point-estimate −0.0034pp. A percentile bootstrap on that 1-of-301 signal returns **p=0.742** — the direction is consistent across taus but the magnitude does not survive sampling variation. The stage is formally inert on this fixture.

Mechanism (locked). The §94c lift is *one* mechanism, not a pipeline of mechanisms: episode → L0 promotion via EpisodeExtraction. The downstream stages are, on LoCoMo10, operationally inert with one borderline exception — `appraisal` reranking (S6→S7 suffix-CI) costs Δgold_recall@k −0.0075pp [−0.017, −0.001] p=0.038, displacing more golds than it surfaces (5 lost-rank-1 vs 1 gained-rank-1, salience gap +0.033 [+0.017, +0.050] p=0.001). A category breakdown localizes the damage to category 5 (multi-hop / open-ended). A bounded-cap intervention (appraisal_salience_cap=0.30) was promising at point-estimate but did not survive its own paired bootstrap (Δhit@1 p=0.399 overall, p=0.740 on the multi-hop slice).

Consequence for the architecture claim. Engram's §87 pipeline is a *scaffold for safe memory mutation*, not a stack of independently contributing retrieval boosters. The defensible v0.2 claim is the narrower one: **on a Mem0-shaped extraction-and-store benchmark, episode-to-L0 promotion is the entire retrieval mechanism**, and the remaining stages earn their place on lifecycle, audit, and property-test grounds (§A4.2) rather than on hit@1.

A.1.1 Churn-budget sweep — `schema_promote_threshold` is operationally inert on LoCoMo10

Source: bench/results/locomo_promote_threshold_sweep.json. Driver: evals/locomo_promote_threshold_sweep.py. Wall: 199 s. n_pairs=301 (max_instances=2, synthesis on, paired bootstrap 2000).

The §87 schema-lifecycle controller exposes a promote/deprecate/recover threshold trio (ConsolidationConfig.schema_promote_threshold default `3`, exposed in commit `dac4f5f`). Lowering the promote threshold makes schema creation *easier*; raising it makes the SCHEMA table sparser. To test whether the recall lift is sensitive to this churn budget, we swept t ∈ {1, 2, 3, 5, 7, 10} on the §94c paired harness with §93 synthesis enabled (so any threshold-driven schema would actually fire).

Every column is bit-identical across the grid. The churn proxy (schemas_created per consolidation tick) holds flat at 1.50, because LoCoMo's per-sample event volume produces only 2–3 candidate super-schemas total — *all* of them either reach the promote bar at `t=1` or fall below it at every `t`. There is no support density in the operating regime where the threshold can bite.

This is the second falsification of the §87 schema-family attribution, matched to §A.4.6's stage-LOO result: the gate's *budgeting* knob is just as inert as the gate's *gating* knob on this fixture. Any future recall claim that depends on schema_promote_threshold needs a fixture where the candidate-schema count is at least an order of magnitude denser than LoCoMo10 produces. The threshold is documented and configurable for downstream users, but on the v0.2 paper benchmark it is a no-op.

Full-10 reconfirmation. We re-ran the same sweep at the §94c operating point with max_instances=10 (n_pairs=1978, resamples=2000, wall=1586 s; source bench/results/locomo_promote_threshold_sweep_full10.json). The inertness is bit-identical at higher fixture density: every cell yields Δhit@1 = +0.0657 [+0.0526, +0.0794], Δhit@k = +0.0718 [+0.0536, +0.0905], Δgold_recall@k = +0.0694 [+0.0512, +0.0871], with schemas_created flat at 1.20 per sample for all t ∈ {1,2,3,5,7,10}. The 5× scale-up tightens the CIs (≈0.027 wide vs ≈0.080 at n=2) and shifts the point estimate from +0.0731 → +0.0657 — both effects expected from the larger sample — but does not perturb the threshold ranking. The "operationally inert on LoCoMo" verdict therefore holds across two independent fixture sizes; the redesign-needed claim above stands.

t	Δhit@1 [95% CI]	Δhit@k	Δgold_recall@k	schemas_created/sample
1	+0.0731 [+0.033, +0.113]	+0.1495	+0.1427	1.50
2	+0.0731 [+0.033, +0.113]	+0.1495	+0.1427	1.50
3	+0.0731 [+0.033, +0.113]	+0.1495	+0.1427	1.50
5	+0.0731 [+0.033, +0.113]	+0.1495	+0.1427	1.50
7	+0.0731 [+0.033, +0.113]	+0.1495	+0.1427	1.50
10	+0.0731 [+0.033, +0.113]	+0.1495	+0.1427	1.50

A.1.2 Dense-fixture reconfirmation — `schema_promote_threshold` inert on `synth_entity`

Source: evals/results/synth_entity_promote_threshold_sweep.json. Driver: evals/synth_entity_promote_threshold_sweep.py. Wall: 556 s. Fixture: 1 760 mems, 1 280 colliding queries (n_entities=32, K=8), vector_weight=0.3, `embed=hash`, paired bootstrap 5 000.

The §A.4.6.1 inertness verdict was based on LoCoMo10, where only 2–3 candidate super-schemas exist per sample. To rule out a fixture-density artifact, we re-ran the sweep on synth_entity (the synth_entity fixture is documented in bench/reports/ner_investigation_report.md §A.4.13b) — a dense, purely entity-collision fixture engineered specifically so the schema-creation path has ample support to fire. We swept the same grid t ∈ {1, 2, 3, 5, 7, 10} with paired baseline (consolidate-off) / treatment (consolidate-on) arms, and report paired Δ vs the `t=3` pivot.

Every metric (h@1, h@5, MRR) and the churn proxy (schemas_created=5) is bit-identical across the entire `t` grid. Paired Δ vs `t=3` is +0.0000 with [+0.0000, +0.0000] CI on every metric × every threshold. Even on a fixture explicitly designed to maximize candidate-schema support density — 32 entities × 8 collisions/entity with bridge queries — the promote/deprecate/recover gate produces an identical SCHEMA-table population at every threshold setting in the swept range.

Combined with §A.4.6.1, this is the *third* independent falsification of the hypothesis that schema_promote_threshold modulates retrieval quality on any v0.2 fixture: LoCoMo10-n2, LoCoMo10-n10, and the dense synth_entity fixture all agree the knob is mechanically inert. Either the candidate-set arithmetic in _promote_candidates saturates below `t=1` and clips above `t=10`, or the gate is a no-op by construction on these inputs. Either way, the v0.2 paper does not depend on the threshold being tuned, and any future claim that does will need a fixture engineered to actually exercise the promote/deprecate boundary — which synth_entity notably fails to do despite being engineered for exactly that.

A.2 PRF × share_prior — five-axis CI panel and joint α × n_pairs surface

Source: PAPER_NOTES anchors 17–31. Drivers: evals/prf_x_shareprior_{stack,gate,breadth,noise,alpha,scale,topk,alpha_scale,grid}_ci.py. Outputs: evals/results/prf_x_shareprior_*_n10*_ci.json.

§A4.3 introduced PRF (entity-based query expansion) and `share_prior` (rank-prior reranker, α-scaled boost capped to non-#1 candidates) as two retrieval-time interventions. This section reports the empirical defense of the v0.2 default operating point — dominance_gate d=0.3, max_entities=4, top_k_for_prf=10, α=`0.05`, on `n_pairs=60`, plain_distractors=80 — across six orthogonal sweep axes, each with n=10-seed paired-bootstrap CIs (5 000 resamples per cell, shared seed indices across the four 2×2 cells per draw so the interaction term is honest).

A.2.1 Headline 5-axis CI matrix (bridge pair@10 interaction)

The interaction term (Δ_BOTH − (Δ_PRF + Δ_SP)) measures how much of the stack's lift is super-additive (i.e. the two interventions co-repair errors that neither catches alone). Across five axes:

Headline. At the default operating point, bridge pair@10 interaction = **+0.075 [+0.060, +0.092] p<0.001** with do-no-harm on unique-fact hit@1 (+0.012 [+0.004, +0.021] p=0.005). Each axis exposes a known, CI-distinguished failure mode (gate ≥ 0.4 collapses to additive; $\alpha \geq 0.10$ trades

t	base h@1	treat h@1	base h@5	treat h@5	base MRR	treat MRR	schemas
1	0.036	0.038	0.233	0.223	0.127	0.125	5
2	0.036	0.038	0.233	0.223	0.127	0.125	5
3	0.036	0.038	0.233	0.223	0.127	0.125	5
5	0.036	0.038	0.233	0.223	0.127	0.125	5
7	0.036	0.038	0.233	0.223	0.127	0.125	5
10	0.036	0.038	0.233	0.223	0.127	0.125	5

Axis	Level	Interaction Δ-of-Δ	95% CI	p	Regime
gate	d = 0.1	+0.090	[+0.070, +0.113]	<0.001	super-additive
gate	d = 0.2	+0.090	[+0.070, +0.113]	<0.001	super-additive
gate	**d = 0.3 (default)**	**+0.075**	[+0.060, +0.092]	<0.001	super-additive
gate	d = 0.4	+0.025	[−0.003, +0.050]	0.091	additive (n.s.)
gate	d = 0.5	+0.000	[−0.000, +0.000]	1.00	inert
breadth	me = 1	+0.030	[+0.000, +0.065]	0.048	weak super-add
breadth	me = 2	+0.090	[+0.070, +0.110]	<0.001	super-additive
breadth	**me = 4 (default)**	**+0.072**	[+0.057, +0.087]	<0.001	super-additive
breadth	me = 8	+0.072	[+0.057, +0.087]	<0.001	super-additive
noise	pd = 40	+0.087	[+0.067, +0.107]	<0.001	super-additive
noise	**pd = 80 (default)**	**+0.070**	[+0.050, +0.090]	<0.001	super-additive
noise	pd = 160	+0.102	[+0.082, +0.122]	<0.001	super-additive
noise	pd = 320	+0.105	[+0.085, +0.125]	<0.001	super-additive
alpha	α **= 0.05 (default)**	**+0.075**	[+0.060, +0.092]	<0.001	super-additive
alpha	α = 0.10	(collapses)	brackets 0	n.s.	additive (CI 0)
alpha	α = 0.20	strictly null	≈ 0	n.s.	inert
topk	k = 5	**+0.102**	[+0.052, +0.155]	<0.001	super-add (BAD)
topk	**k = 10 (default)**	**+0.070**	[+0.050, +0.092]	<0.001	super-additive
topk	k = 20	−0.007	[−0.018, +0.003]	0.18	additive (n.s.)
topk	k = 40	−0.007	[−0.018, +0.003]	0.18	additive (n.s.)
scale	n_pairs = 30	−0.200	(CI excl. 0)	<0.001	sub-additive
scale	**n_pairs = 60 (default)**	**+0.075**	[+0.060, +0.092]	<0.001	super-additive
scale	n_pairs = 120	−0.053	(CI excl. 0)	<0.001	sub-additive
scale	n_pairs = 200	+0.059	(CI excl. 0)	<0.001	super-additive

bridge gain for unique-fact regression; top_k_for_prf=5 catastrophically regresses unique hit@1 −0.200 [−0.214, −0.188] p<0.001) that the defaults are chosen to fence off.

A.2.2 No-regression on absolute lift (Δ_BOTH bridge pair@10)

The interaction sign flips with corpus size (PRF-alone strengthens at large n), but the absolute stack lift Δ_BOTH stays strictly positive at every scale tested. **The stack does not regress as the corpus grows.**

A.2.3 Joint $\alpha \times$ n_pairs surface (anchors 27 + 31)

To rule out marginal-cut artifacts on the α and scale axes, we mapped the joint $\alpha \in \{0.05, 0.10, 0.20\}$ $\times$ n_pairs $\in \{30, 60, 120, 200\}$ surface at n=10 seeds with 10 000 paired bootstrap resamples per of the 12 cells.

Bridge pair_recall@10 — interaction (Δ-of-Δ) 95% CI. Each cell is the paired-bootstrap CI on the super-additivity term Δ_BOTH − (Δ_PRF + Δ_SP) with shared seed-resample indices across the four arms (C0, CP, CR, CB) per draw:

α=0.05 ties or wins on interaction at every column, including both super-additive scales (n_pairs $\in \{60, 200\}$) and both sub-additive scales (30, 120) where it is the *least* sub-additive. At n_pairs=200

n_pairs	Δ_BOTH bridge pair@10	95% CI	p
30	+0.230	(CI excl. 0)	<0.001
60	+0.095	[+0.060, +0.130]	<0.001
120	(positive)	(CI excl. 0)	<0.001
200	+0.095	(CI excl. 0)	<0.001

α \ n_pairs	30	60	120	200
0.05	−0.150 [−0.200, −0.100] p<0.001	**+0.070 [+0.050, +0.092] p<0.001**	−0.042 [−0.061, −0.022] p<0.001	**+0.061 [+0.048, +0.073] p<0.001**
0.10	−0.185 [−0.220, −0.155] p<0.001	−0.020 [−0.043, +0.003] p=0.12	−0.125 [−0.160, −0.089] p<0.001	+0.020 [+0.003, +0.037] p=0.025
0.20	−0.050 [−0.080, −0.015] p=0.004	−0.020 [−0.052, +0.007] p=0.22	−0.070 [−0.086, −0.056] p<0.001	+0.028 [+0.016, +0.039] p<0.001

all three α settings produce CI-clean super-additive interactions but α=0.05 still dominates: largest interaction (+0.061), strict CI exclusion of zero, and the largest unique do-no-harm gain (+0.009 vs −0.020/−0.024 at α=0.10/0.20). Three findings, in order of paper relevance:

(i) **α=0.05 ties or wins on interaction at every scale**, including the two sub-additive scales (n_pairs ∈ {30, 120}) where it is the *least* sub-additive. No (α, n_pairs) cell exists where relaxing α to 0.10 or 0.20 recovers super-additivity that α=0.05 doesn't already provide.

(ii) **Δ_BOTH absolute lift is strictly positive in every cell of the 3 × 4 grid** (12/12 cells, min +0.042, max +0.367). The interaction sign-flip is a story about how much of the lift is super-additive vs. additive, not about whether the lift exists.

(iii) **Unique do-no-harm hit@1 is a function of α only**: 0.842 at α=0.05, 0.808 at α=0.10, 0.804 at α=0.20 — corpus-size invariant. This rules out the α=0.10 unique-fact regression being an n_pairs artifact.

Verdict. α=0.05 is the only Pareto-optimal point on the joint α × n_pairs surface, ties or wins on Δ_BOTH absolute lift, and strictly wins on unique do-no-harm. Anchors 25/26's individual findings are not artifacts of the marginal cuts — they hold on the full joint surface.

A.2.4 PRF expansion budget (`top_k_for_prf`) sweep

`k=5` is the single sharpest Pareto-rejection in the §A4.3 corpus: a choice of operating point by bridge-interaction alone would land on +0.102 super-additive interaction while silently destroying unique hit@1 by 19–21 absolute points. `k=10` is uniquely Pareto-optimal — it ties `k=20`/`k=40` on unique do-no-harm at the maximum (0.828) and beats them on bridge interaction. Anchors 27 and 30 together close the two axes a reviewer might propose to "improve" the headline numbers (relaxing α; tightening k).

A.2.5 Paper-ready summary

PRF × share_prior super-additivity is robust across six orthogonal sweep axes (PRF dominance gate d ∈ [0.1, 0.5], breadth me ∈ {1, 2, 4, 8}, distractor density pd ∈ {40, 80, 160, 320}, share_prior weight α ∈ {0.05, 0.10, 0.20}, corpus scale n_pairs ∈ {30, 60, 120, 200}, and PRF budget top_k_for_prf ∈ {5, 10, 20, 40}). At the default operating point (d=0.3, me=4, α=0.05, pd=80, n_pairs=60, k_prf=10), bridge pair@10 interaction = +0.075 [+0.060, +0.092] p<0.001 (n=10 seeds, paired bootstrap 5 000 resamples) with do-no-harm on unique-fact hit@1 (+0.012 [+0.004, +0.021] p=0.005). Each axis exposes a known, CI-distinguished failure mode that the defaults are chosen to fence off. Δ_BOTH absolute lift is strictly positive at every (α, n_pairs) cell of the joint surface (12/12 cells, [+0.042, +0.367]).

Total evaluator wall amortized over six cron-run anchors: ~110 min for the 5-axis n=10 panel + 1 170 s for the joint α×n_pairs grid + 268 s for the n=10 top_k axis. Stats helpers covered by 17 unit + property tests in tests/evals/test_{stack,axis}_ci.py.

A.2.6 Adaptive α — opt-in regularizer for over-shot α

§A7.2 introduces the optional α_eff = α / (1 + max(0, max_deg − 1)/4) schedule behind RetrievalConfig.share_prior_adaptive_alpha. The question is empirical: does tapering buy lift, hurt lift, or trade regimes?

We A/B'd constant vs adaptive α on the bridge corpus, 3 seeds, at α ∈ {0.05, 0.10, 0.20, 0.40}

k_prf	Δ_BOTH (bridge @10)	interaction	unique hit@1 (CB) Δ
5	+0.152 [+0.087,+0.217] p<0.001	**+0.102** [+0.052,+0.155] p<0.001	**0.619** (Δ −0.200 [−0.214,−0.188] p<0.001)
10	+0.080 [+0.043,+0.117] p<0.001	**+0.070** [+0.050,+0.092] p<0.001	0.828 (Δ +0.009 [0.000,+0.018] p=0.07)
20	+0.020 [+0.007,+0.033] p=0.007	−0.007 [−0.018,+0.003] p=0.18	0.828
40	+0.020 [+0.007,+0.033] p=0.007	−0.007 [−0.018,+0.003] p=0.18	0.828

(driver: evals/share_prior_adaptive_alpha.py). The result is regime-flipped:

Reading. Adaptive α is a *hedge against* α *over-shoot*, not a free lift. At the defended operating point (α=0.05) it strictly under-boosts relative to the constant schedule, so it ships **default-off** (share_prior_adaptive_alpha=False). The use case is operators who auto-tune α across heterogeneous corpora: when α drifts up to 0.20, the constant schedule's boost saturates the rank-0 cap on multiple candidates and regresses to baseline; the adaptive schedule shrinks the boost in proportion to pool density and keeps the bridge-fact lift intact. Adaptive α is therefore a **robustness knob for** α **auto-tuners**, not a default-on improvement.

A.2.7 Mechanism: share_prior as a PRF-conditional repair signal

§A.4.7.1's interaction CIs answer *whether* the stack is super-additive; they do not pin *why*. Two cells in the breadth and gate sweeps make the mechanism legible.

Breadth `me=2` — the cleanest mechanistic anchor. At max_entities=2, PRF alone regresses bridge pair@10 (Δ_PRF is negative on point estimate, dragged below baseline by a single wrong-entity expansion that admits distractors), yet Δ_BOTH is positive: the stack lifts despite PRF acting as a net negative on its own. In CI form (n=10), the me=2 interaction is +0.090 [+0.070, +0.110] p<0.001 — the largest super-additive cell on the breadth axis, *driven by share_prior repairing PRF's expansion error inside the pool*. share_prior is therefore not an independent reranking lift (Δ_SP ≈ 0 across every breadth) but a **PRF-conditional repair signal**, and its value is largest precisely where PRF is most likely to err. At me=1 PRF is too conservative to make wrong-entity errors, so SP has no PRF-induced damage to repair (interaction CI [+0.000, +0.065] crosses 0 at the edge); at me $\geq$ 4 the pool is large enough that PRF mistakes are diluted but SP can still rescue them (CI [+0.057, +0.087] excludes 0).

Dominance gate d=0.2 → d=0.3 is a hard safety floor, not a knob. The unique-fact hit@1 CIs at d=0.2 and d=0.3 are *non-overlapping by ~50pp*: at d=0.2, Δ_BOTH = −0.475 [−0.487, −0.463] p<0.001 (catastrophic — PRF fires on single-hop queries where no entity genuinely dominates, expansion drags in distractors, SP cannot rescue *the wrong query*); at d=0.3 it reverses to Δ_BOTH = +0.017 [+0.013, +0.025] p<0.001. The cliff is a property of the regime, not seed noise. The takeaway is that min_dominance is a hard safety boundary: below 0.3, the bridge gain (interaction +0.090–+0.100) is paid for with a 47-point single-hop hit@1 catastrophe. The §A.4.7 defaults fence both sides — d≥0.3 to preserve single-hop, k_prf≥10 to fence the symmetric Pareto trap on the unique side (§A.4.7.4).

Together these two cells convert the §A.4.7.1 statistical statement ("the interaction is super-additive at the default operating point") into a mechanistic one ("PRF widens the candidate net at a controlled single-hop cost; share_prior repairs the wrong-entity expansions PRF makes inside the widened net"). They also explain why the two interventions ship as a stack rather than as independent flags: SP alone is empirically inert across every breadth and budget level in the panel, and its value is unlocked only by PRF-induced pool disturbance.

A.3 Moved: §A.4.8.1 (LongMemEval treatment-arm Δ tables)

The full per-arm × per-type × per-metric Δ matrices for the n=500 LongMemEval treatment arms have been moved to **bench/reports/treatment_arm_dumps_report.md**. Headline result and per-type panel summary live in §4.6.

A.4 LongMemEval — sentence-transformer embedding headline (n = 100)

To check whether the §4.6 hash-trigram floor leaves real lift on the table when the dense channel is a real semantic encoder, we re-ran the LongMemEval-S baseline arm on the first n = 100 instances with the sentence-transformer (all-MiniLM-L6-v2, 384-d) embedder wired in via `--embed st` and the post-§4.5 default vector_weight = 0.3.

α	Δpair@10 (adaptive − constant)	Reading
0.05	−0.077	tapering under-boosts (safe regime)
0.10	−0.154	tapering under-boosts (safe regime)
0.20	**+0.154**	rank-0 cap saturates on dense pools; tapering recovers signal
0.40	~0	both arms collapse to baseline

Headline (n = 100, k = 10, embed = ST, vw = 0.3):

Per question type (n = 100 slice):

The n = 100 slice is dominated by single-session-user (70%) and multi-session (30%) because the public LongMemEval-S release lays those types down first. Comparing to the §4.6 hash-trigram baseline on its full n = 500 panel (per-type cells, not the 100-prefix):

The two cells aren't size-matched (the §4.6 cells are n=70 and n=133; this one is n=70 and n=30), so this is a directional read, not a paired CI. The ST encoder *appears to help* the multi-session cell (+9.5 pp absolute hit@1) where the answer turn is rarely a lexical hit and lives many turns from the question, but *hurts* the single-session-user cell (−7.1 pp) where BM25 already finds the right session at 91.4% and the dense channel adds noise. This is consistent with the §4.5 vw-Pareto finding that the dense channel's marginal value is corpus-shape dependent. The ingest cost is ≈ 26× the hash baseline (13.8 s/inst vs 523 ms/inst in §4.6) — almost entirely the MiniLM forward pass over ~500 turns/instance — while recall p50 only doubles (23 ms vs 10 ms), because the vector ANN scan is still cheap relative to BM25 at this N. The full n = 500 ST sweep is deferred until we have a per-cell vw optimization story; on the 100-prefix, ST is a wash overall (0.860 vs the §4.6 100-prefix overall of ~0.876), masking a real per-type trade-off that §A7.2 and §A7.3 should be evaluated against in a follow-up. Artifact: bench/results/lme_n100_st_vw0.3_baseline.json. Reproduce: python -m evals.longmemeval_adapter --max-instances 100 --k 10 --arm baseline --embed st --vector-weight 0.3 --out bench/results/lme_n100_st_vw0.3_baseline.json. Wall ≈ 24 min on the cron host (single-process, nice 5).

A.4.1 Cross-check: legacy `vector_weight = 0.5` on the same n = 100 ST slice

To make sure §4.5's vw=0.3 default carries over to the ST embedder on LongMemEval, we mirrored the run above with the legacy default vector_weight = 0.5, all other knobs identical (bench/results/lme_n100_st_vw0.5_baseline.json).

The two arms are paired-by-question (same 100 instances, same haystack, same encoder, same k); the only delta is the BM25↔ANN convex weight. Headline overall hit@1 moves +0.010 (vw=0.3 better) and hit@10 moves +0.010 in the same direction. With n=100 and a per-question Bernoulli under H : p=0.5 on flips, that gap is well inside noise — a binomial 99% CI on a single point estimate at n=100 is roughly ±0.090 — but it is *consistent in sign* with the §4.5 hash-channel Pareto and so we keep vector_weight = 0.3 as the package default for both embedders. We deliberately do *not* claim a real ST-specific lift from this slice; the ST default is a *do-no-harm* call relative to the already-defended hash default.

Cost is unchanged across the two arms (ingest dominated by the MiniLM forward pass, recall dominated by the ANN scan — neither depends on vw). Reproduce: python -m evals.longmemeval_adapter --max-instances 100 --k 10 --arm baseline --embed st --vector-weight 0.5 --out bench/results/lme_n100_st_vw0.5_baseline.json.

A.4.2 Embedder × `vector_weight` 2×2 on the same n = 100 slice

To complete the embed × vw cell story we mirrored the hash channel at the new default vector_weight = 0.3 on the same 100 LongMemEval-S instances and the same 49 878-memory haystack (bench/results/lme_n100_hash_vw0.3_baseline.json). All four cells are *paired by question* (identical instance subset, identical k=10).

Reading. ST dominates hash on hit@1 by **+0.160** (0.860 vs 0.700), with the gap concentrated almost entirely in multi-session questions (ST 0.900 vs hash 0.567, n=30) — exactly the regime where dense recall is supposed to help, since the hash channel cannot generalize across paraphrase across sessions. On single-session-user (n=70) the gap narrows to +0.086 (0.843 vs 0.757). At hit@10 both embedders converge to 0.95–0.96 — the ST advantage is a *ranking* advantage, not

metric	value
session_hit@1	**0.860**
session_hit@10	**0.960**
n_memories_total	49,878
ingest p50 / inst	13,830 ms
recall p50 / q	23.1 ms

type	n	hit@1	hit@10
single-session-user	70	0.843	0.957
multi-session	30	0.900	0.967
overall	100	0.860	0.960

a recall-coverage advantage, consistent with the convex-hybrid design where hash provides surface-form coverage and ST provides paraphrase ranking.

Cost. Hash is ~2.4× cheaper at ingest (5.7 s vs 13.8 s p50) and ~1.7× cheaper at recall (13.9 ms vs 23.1 ms p50). For deployments where multi-session paraphrase recall is rare or where 24 ms p50 is unacceptable, the hash channel is a defensible operating point — but for general LongMemEval-shaped traffic, ST's +0.160 hit@1 wins by a wide margin that is well outside the n=100 binomial CI (~±0.090).

The vw=0.3 default holds across embedders: vw=0.5 hurts ST by 0.010 on both hit@1 and hit@10 in the same-instance paired comparison, and the hash channel was already on vw=0.3 from §4.5's Pareto sweep. Reproduce: python -m evals.longmemeval_adapter --max-instances 100 --k 10 --arm baseline --embed hash --vector-weight 0.3 --out bench/results/lme_n100_hash_vw0.3_baseline.json.

A.4.3 LongMemEval — sentence-transformer baseline at n = 500

To check whether the §A.4.8.2 ST headline holds at five times the instance count, we ran the same configuration (embed=ST, vector_weight = 0.3, baseline arm — no PRF, no share_prior) on 500 LongMemEval-S instances. The full haystack is **246 918 memories** across the 500 sessions (mean ≈ 494 mem / instance), an order of magnitude larger than any preceding LME cell in this paper.

The n=100 → n=500 deltas (−0.002 hit@1, −0.010 hit@10) are well inside the n=500 binomial CI (~±0.030 at p=0.86): the ST headline replicates. Per-type at n=500 (paired counts in parentheses):

Multi-session at scale. The §A.4.8.2.2 multi-session ST hit@1 of 0.900 (n=30) lands at **0.895 on n=133** — the paraphrase-across- sessions advantage that motivated the dense channel survives the 5× scale-up, with the larger sample now placing it tightly around 0.90 (CI ~±0.052).

Recall-latency at scale. With 247k memories on disk per instance, recall p50 holds at 25.3 ms — within 10% of the n=100 figure (23.1 ms) — confirming that hybrid recall is dominated by the top-k merge, not by the haystack size, for this regime. Ingest p50 is 14.8 s (vs 13.8 s at n=100); the small drift is mostly the cold ST encoder warm-up amortizing across more sessions, not memory growth in any session.

The matched PRF×share_prior arm at the same operating point (α=0.05, d=0.3, pool=20) is reported in §A.4.8.2.4 below. Reproduce: python -m evals.longmemeval_adapter --max-instances 500 --k 10 --arm baseline --embed st --vector-weight 0.3 --out evals/results/lme_n500_st_vw03_baseline.json.

A.4.4 LongMemEval — paired PRF × share_prior at n = 500 (ST, vw = 0.3)

Same n=500 slice, same ST embedder, same vw=0.3 — paired comparison against the §A.4.8.2.3 baseline at the operating point we defended in §A.4.7 (α=0.05, d=0.3, pool=20).

Reproduce: python -m evals.longmemeval_adapter --max-instances 500 --k 10 \{} --arm both --embed st --vector-weight 0.3 \{} --sp-alpha 0.05 --sp-pool 20 --qe-dominance 0.3 \{} --out evals/results/lme_n500_st_vw03_prfsp.json.

Aggregate (paired, n=500).

Per-type Δhit@1 (paired bootstrap, 2k iter, seed=2026).

cell	hash@vw=0.3 (§4.6 n=500)	ST@vw=0.3 (n=100)	Δhit@1
single-session-user	0.914	0.843	−0.071
multi-session	0.805	0.900	+0.095

vw	session_hit@1	session_hit@10	sss-user h@1	multi-session h@1
0.3	**0.860**	**0.960**	0.843	**0.900**
0.5	0.850	0.950	0.829	**0.900**

Interpretation. PRF×share_prior at the §A.4.7 operating point is a **net negative at n=500**: the aggregate Δhit@1 95% CI is [−0.042, −0.002], excluding zero. The aggregate Δhit@10 CI also excludes zero. The damage concentrates on single-session-user and temporal-reasoning — types where the question is already lexically proximate to a single answering session, so PRF's pseudo-relevance expansion drags top-k toward neighbour sessions sharing the query vocabulary. knowledge-update, single-session-assistant, and single-session-preference are flat (CI brackets zero). On the latency axis, PRF×SP costs +1.85× p50 and +4.5× p99 (the tail includes spaCy NER warm-paths on long sessions).

Decision. Ship RetrievalConfig.query_expansion_min_dominance = None as the default — i.e. **PRF×SP off by default** (decision #2 in the v0.2 plan, now empirically defended). The knob remains runtime-toggleable for corpora where the §A.4.7 multi-entity-hard / adversarial regimes apply. We re-evaluate when the real LoCoMo dataset lands; the synthetic-LoCoMo placeholder (SCALE_REPORT §D7) also shows PRF×SP regression, suggesting the LongMemEval result is not a single-corpus artifact.

Default actually flipped (2026-05-24). Previously RetrievalConfig.query_expansion_min_dominance defaulted to `0.3` (ON) under a draft v0.3 operating point. With the §A.4.8.2.4 paired n=500 CI now in the camera-ready, the dataclass default is **`None` (OFF)** and locked by tests/unit/test_v0_3_defaults_locked.py. Anchor-share gate value remains 0.5 so opt-in (.min_dominance = 0.3) immediately activates the §D15d-style operating point with one knob. Adversarial / unit suites (1360 tests) green at OFF default.

Determinism replication (2026-05-24). A second n=500 run with the v0.3 defaults wired (vector_weight=0.3 from the dataclass default, qe_dominance=0.3, sp_alpha=0.05, `sp_pool=20`, qe_anchor_share_max=0.5, ST embedder) produced session_hit@1=0.836 and session_hit@10=0.938 — bit-exact match to the original PRF×SP column above. Recall p50=47.04 ms vs. 46.79 ms (within process noise). Result file: evals/results/lme_n500_st_vw03_prfsp_v03defaults.json. This pins the operating point: switching from explicit CLI overrides to dataclass defaults does not perturb the headline number, so the v0.3-defaults flip in §A.4.5.1 carries the same evidence as the original sweep and the regression CI [−0.042, −0.002] holds verbatim.

A.4.5 Per-type Δhit@5 breakdown (paired, n = 500)

§A.4.8.2.4 reported per-type Δhit@1 only. For symmetry and to localize where the −0.012 aggregate hit@5 regression lives, we compute the matching paired-bootstrap CIs on hit@5 from the same JSONs (evals/results/lme_n500_st_vw03_{baseline,prfsp}.json, R=2000, seed=2026):

The aggregate hit@5 regression is **entirely concentrated** in single-session-user (point −0.043, CI right edge at zero) and temporal-reasoning (point −0.023, CI overlaps zero). Four of the six types are exactly flat on hit@5 — PRF×SP changes nothing for them at this operating point. The same two types also drove the §A.4.8.2.4 hit@1 regression, so the failure mode is single-axis: when the question is already lexically proximate to one answering session, PRF expansion drags top-k toward neighbours sharing query vocabulary. This strengthens the §A.4.8.2.4 ship decision (query_expansion_min_dominance = None by default) by showing the damage is type-localized rather than diffuse — a future type-aware gate (§A.4.10) is the right remediation surface.

embed	vw	session_hit@1	session_hit@10	sss-user h@1	multi-session h@1	ingest p50	recall p50
ST	0.3	**0.860**	**0.960**	0.843	**0.900**	13.8 s	23.1 ms
ST	0.5	0.850	0.950	0.829	**0.900**	13.9 s	23.1 ms
hash	0.3	0.700	0.950	0.757	0.567	5.7 s	13.9 ms

n	embed	vw	session_hit@1	session_hit@10	ingest p50	recall p50
100	ST	0.3	0.860	0.960	13.8 s	23.1 ms
500	ST	0.3	**0.858**	**0.950**	14.8 s	25.3 ms

A.5 Multi-entity-hard fixture (D1 v0.3) — out-of-distribution check on PRF/share_prior

LongMemEval-S is a single corpus shape with a single answer distribution. To test whether the §A.4.7/§4.6 PRF and share_prior behaviors generalize, we built a non-saturated multi-entity-collision fixture (evals/corpora/multi_entity_hard.py). Each fact is a short PERSON×ORG or PERSON×LOC triple ("Alice works at Apple."); each fact is paired with N type-collision distractors that re-use the gold entity surface form in a *different sense* ("Alice ate an apple at lunch.", "Alice has a coworker named Jordan."). High-overlap distractors that share the query verb without an entity hit are also planted. The fixture is reproducible under fixed seed and the hardness contract (BM25-like hit@5 < 0.7) is asserted in unit tests (tests/evals/test_multi_entity_hard.py).

Headline (n_facts=500, n_sessions=25, distractors_per_fact=8, 5000-memory haystack, k=10, default v0.2 hybrid retriever, 3 seeds $\in$ {1, 2, 3}, paired bootstrap, 5000 resamples, n=1500 paired queries):

Bold cells exclude 0 at 95%.

Reading. With three independent seeds and CIs, the regression sharpens: PRF alone is **CI-confirmed neutral-to-negative** (Δhit@5, Δhit@10 strictly < 0); share_prior alone gives a small but **CI-significant +0.016 hit@1** lift, with hit@10 modestly negative (CI excludes 0); the stacked "both" arm collapses hit@10 by $-$**0.124 [$-$0.147, $-$0.102] (p<0.001 against H =0)** — the worst arm in deep recall by a wide CI-non-overlapping margin. The same qualitative pattern observed on LongMemEval-S (§4.6) repeats on a corpus with entirely different surface forms and answer distribution, with paired CIs that exclude 0 — strong evidence that the v0.2 defaults (query_expansion_min_dominance=None, no share_prior reranker) are not LongMemEval-overfit, and that the regression is mechanistic, not seed noise.

What it doesn't say. This run does *not* invalidate PRF or share_prior — it shows that on corpora where the discriminative signal is *entity-type sense* rather than entity-surface frequency, the current PRF gate (frequency-only dominance) expands in exactly the wrong direction. The remediation, consistent with §A.4.8.1's d-ablation conclusion, is a type-aware PRF gate that only fires when the dominant first-pass entity type is one the corpus actually disambiguates by. That prototype is implemented and evaluated in §A.4.10.

Artifacts: bench/results/multi_entity_hard_arms_d8.json (single-seed point estimates), evals/results/multi_entity_hard_arms_3seed_ci.json (3-seed paired-bootstrap CIs); reproduce via python -m evals.multi_entity_hard_arms --n-facts 500 --n-sessions 25 --distractors-per-fact 8 --seeds 1 2 3 --resamples 5000.

A.6 Moved: §A.4.10 – §A.4.12, §A.4.15 / §A.4.15b–o (excluding §A.4.15j and §A.4.15-profile)

The 17 PRF-falsification subsections that originally lived here have been moved verbatim to **bench/reports/prf_falsification_report.md** as a Phase-3 page-budget triage step (banner at top of this file).

Body-cited summaries retained: - **§A.4.15j** anchor-share gate inertness on LongMemEval (cited from §5.1 as part of the §A.4.15j–o aggregate finding). - **§A.4.15-profile** cProfile hotspot characterization across PRF×SP arms (cited from §A4.4 latency-myth discussion).

The paper headlines for this work are §5.1 (PRF scope of the null result) and §A.4.16.4 (the RM3 arm from AUDIT-D, which supersedes the heuristic series for the v0.2 narrative).

type	n	hit@1	hit@10
single-session-assistant	56	0.911	0.929
multi-session	133	0.895	0.977
knowledge-update	78	0.885	0.923
single-session-user	70	0.843	0.957
temporal-reasoning	133	0.827	0.947
single-session-preference	30	0.700	0.933

metric	baseline (§A.4.8.2.3)	PRF×SP	Δ (PRF×SP − base)	95% paired bootstrap CI
hit@1	0.8580	0.8360	**−0.0220**	**[−0.0420, −0.0020]**
hit@10	0.9500	0.9380	**−0.0120**	**[−0.0220, −0.0020]**
recall_ms p50	25.3	46.8	+21.5 ms (+1.85×)	—
recall_ms p99	53.1	238.8	+185.7 ms	—

A.7 Moved: §A.4.13 – §A.4.13i (entity-channel × NER backend investigation)

The 9 entity-channel / NER-backend ablations have been moved verbatim to **bench/reports/ner_investigation_report.md**. The investigation closed as a measured null: small spaCy-NER lift on synthetic synth_entity does not survive a sentence-transformer embedder swap, and `spacy_md` does not reopen the gap. Headline implication is folded into §6 (Threats — single embedder per family) and §A5.1.

A.8 Matched ingest curves — 10k → 100k → 1M

Single-harness ingest scaling. Same tests/scale/ test_ingest_* code path with max_events_per_ minute=0, fresh tmpdir, single writer, SQLite + JSONL backing store; latency is per- Engram. remember wall time.

No knee. Across two orders of magnitude every percentile up to p99 stays sub-4 ms; p50 is flat between 100k and 1M (+0.08 ms). p99 grows just 13% (3.279 → 3.695 ms). 1M-specific p99.9 = 20.83 ms is the SQLite-checkpoint window, not a structural cost (§D5). Throughput peaks at 100k (1.7k w/s) and falls 28% by 1M as checkpoint pressure rises. Reproduce: python -m bench.plot_ingest_curves. The §4.6 verdict cites this section as the testbed-scales-to-production-ingest evidence.

A.9 Reproducibility band on the 1M-ingest curve

The §A.4.14 1M point was re-run six times across six code states on `d9608e1` (SCALE_REPORT.md §D8) to characterise the noise floor. Across runs: **p50 ∈ [0.434, 0.487] ms** (±5.6%), **p99 ∈ [3.355, 3.695] ms** (±4.9%). The tail/head p99 ratio over the six runs is 0.957 — there is no degradation cliff on long-running ingest, and the §A.4.14 single-point percentiles all fall inside the reproducibility band of their respective metrics.

A.10 Moved: §A.4.14r / §A.4.14r-stratified (matched recall-latency)

Supplementary latency curves and the stratified needle-in-haystack recall sweep at 100k → 1M have been moved to **bench/reports/ latency_curves_extra_report.md**. The 1M cross-run reproducibility band (§A.4.14b) remains in this appendix because §A.4.14 and §7 cite it.

A.11 A.4.15-profile Hotspot characterization across PRF×SP arms (cProfile, n=30 k)

A NEXT.md-staged claim that PRF "doubled recall p50" was traced to a noisy single-shot microbench. We re-measured under controlled conditions (scripts/profile_recall_prf_sp.py, n=30 000 memories, q=200 paired queries, seed=42, embed=HashTrigram-256, vector_weight=0.3, OMP_NUM_THREADS=MKL_NUM_THREADS=1; per-arm cProfile, wall-clock ingest, per-query timer for latency to exclude profiler overhead).

PRF and share_prior alone are within sampling noise of baseline (p50 deltas ≤1 ms; p95 deltas

type	n	mean Δ	95% CI
knowledge-update	78	+0.0000	[−0.039, +0.039]
multi-session	133	−0.0226	[−0.068, +0.023]
single-session-assistant	56	+0.0000	[+0.000, +0.000]
single-session-preference	30	+0.0000	[−0.100, +0.100]
single-session-user	70	−0.0571	[−0.129, +0.000]
temporal-reasoning	133	−0.0301	[−0.068, +0.008]

type	n	baseline	PRF×SP	mean Δhit@5	95% CI
knowledge-update	78	0.923	0.923	+0.0000	[+0.000, +0.000]
multi-session	133	0.977	0.977	+0.0000	[+0.000, +0.000]
single-session-assistant	56	0.929	0.929	+0.0000	[+0.000, +0.000]
single-session-preference	30	0.933	0.933	+0.0000	[−0.000, +0.000]
single-session-user	70	0.957	0.914	**−0.0429**	[−0.100, +0.000]
temporal-reasoning	133	0.947	0.925	−0.0226	[−0.060, +0.008]
AGGREGATE	500	0.950	0.938	**−0.0120**	**[−0.024, −0.002]**

≤5 ms, both directions); the only arm with real overhead is `both` (+14 % p50, +24 % p95). cProfile hotspots show sqlite3.Connection.execute accounts for ~73 % of `recall` cum time across *all* arms — SQLite is the floor, not PRF. PRF doubles engine.search call count (200 → 400) but adds only ~6 % to engine.search cum time because the second pass hits warm pages. The v0.3 candidate-pool prune is justified for the `both` arm only; PRF-only and SP-only have no latency caveat. Full hotspot table in SCALE_REPORT.md §A.4.15-profile.

A.12 IDF-rarity filter on PRF candidates (falsified)

The §A.4.15-profile result removed the latency objection to PRF-only; the only remaining barrier to making PRF default-on is the §D15c multi-token-anchor regression, where Δh@1(gated_pref − baseline) falls to roughly −9 pp at answer_anchor_tokens=3 on the synthetic preference corpus. We tested whether the regression is mediated by *low-IDF* (corpus-common) expansion candidates: PRF appends the most frequent novel entities from the top-K pool, but if those entities are themselves common across the active corpus, BM25 scores dilute toward distractors and away from the answer anchor.

Implementation. RetrievalConfig.query_expansion_idf_min_rarity: float | None = None (default OFF). When set, expand_query() drops candidate entities whose corpus rarity = 1 − df / N (where df is the number of active+fading FTS-indexed memories matching the entity) falls below the threshold, *before* truncating to max_entities. The rarity lookup is built lazily by RetrievalEngine.\{} _build_prf_rarity_lookup() against the store's FTS5 index and is memoized per PRF expansion. Lenient on lookup errors (treats them as rarity = 0.0, i.e. filter the candidate). Six unit tests cover the inert-when-None, drops-low-rarity, can-empty-result, lenient-on-lookup-exception, end-to-end smoke, and engine rarity-correctness paths.

Test. Single-seed paired sweep (n=240 facts, 2 400 memories, 240 queries, k=10, answer_anchor_tokens=3, the §D15c-mech-2 worst point) with idf_min_rarity ∈ {None, 0.0, 0.3, 0.5, 0.7, 0.9}. Same baseline once; gated_pref re-run per threshold.

Reading. Δh@1 is **identically −9.17 pp at every IDF threshold from 0 through 0.9**, despite the filter demonstrably engaging (a sanity trace shows `idf=None` chooses [pr, deferred] while `idf=0.9` chooses `[deferred]` for the same query). The IDF-rarity hypothesis is **falsified** for this corpus: removing the corpus-common expansion term doesn't recover any hit@1; the regression survives even when expansion contains *only* the rarest available entity.

Mechanism implication. Combined with §D15c-mech (hard-distractor density falsified), §D15c-mech-2 (multi-token-anchor inverted), and now §A.4.15g (IDF-rarity filter inert), the failure mode of PRF on this corpus is **not** me-

arm	hit@1	hit@5	hit@10	Δhit@1 [95% CI]	Δhit@5 [95% CI]	Δhit@10 [95% CI]
baseline	0.077	0.361	0.596	—	—	—
PRF (d=0.30)	0.077	0.326	0.559	−0.007 [−0.016, +0.003]	−0.033 [−0.043, −0.023]	−0.037 [−0.047, −0.028]
share_prior	0.077	0.367	0.570	**+0.016 [+0.005, +0.028]**	−0.007 [−0.024, +0.010]	−0.026 [−0.043, −0.009]
both	0.079	0.301	0.472	−0.007 [−0.026, +0.011]	−0.077 [−0.098, −0.056]	**−0.124 [−0.147, −0.102]**

N	p50 ms	p95 ms	p99 ms	tput w/s
10,000	1.573	3.528	4.461	494.8
100,000	0.406	0.695	3.279	1,702.4
1,000,000	0.487	1.519	3.695	1,230.8

diated by which entities are appended. The remaining live hypotheses are: (i) the appended *positions* in the BM25 query disturb proximity/saturation effects regardless of token choice, or (ii) the share_prior/PRF interaction with the single-session-preference question template induces a query-side distribution shift that surfaces only in the synthetic schema. v0.3 default for query_expansion_min_dominance stays None (off), and query_expansion_idf_min_rarity ships as off-by-default infrastructure. Code: evals/synth_pref_idf_rarity_sweep.py.

A.13 Anchor-share gate: LongMemEval inertness

§A.4.15i shipped the anchor-share gate behind a runtime knob with v0.3 default `0.5` (synth-pref pareto sweet-spot). Before flipping that default, we confirm LongMemEval-S inertness — the *negative* prediction the §A.4.15h mechanism makes on a corpus where anchors are not mass-recycled across facts.

LongMemEval-S full 500-question evaluation, paired bootstrap vs the saved baseline run, k=10:

Reading. The gate is *bit-identically* inert on LongMemEval at all three thresholds — every per-instance hit@1 / hit@k matches raw PRF. The gate fires on **zero** LME queries, including at the synth-pref SE=0 floor (0.4). This is the falsifiable inertness §A.4.15h predicted: LongMemEval anchors are not mass-recycled across facts, so first-pass top-K is never saturated by a single dominant entity.

Implication. Flipping the v0.3 default to anchor_share_max = 0.5 is LongMemEval-safe at the bit level. The single-session-preference slice keeps its +6.67 pp PRF lift (0.3667 → 0.4333) under the gate. The synth-pref −9.17 pp regression is fully cured at threshold 0.4 (§A.4.15i) without touching any LME metric. v0.3 candidate-default tuple: min_dominance = 0.3, anchor_share_max = 0.5, type_allow = None. Code: scripts/run_d15d_lme_sweep.sh, bench/results/lme_full500_d15d_anchor_sweep.json. SCALE_REPORT.md §D15d-LME has the full per-type breakdown.

A.14 BGE-large-en-v1.5 — third embedder tier on the entity-collision protocol

Question. A natural reviewer ask is "why only two embedders?" The two-axis result of §4.3 (HashTrigram-256 vs. ST MiniLM-384, 384-d) could plausibly be an artifact of the dense-side capacity ceiling: a larger encoder might erase the lexical-tag advantage and turn every cell uniformly positive. To falsify this, we re-ran the entity-collision protocol with BAAI/bge-large-en-v1.5 (1024-d) under the v0.2 RetrievalConfig defaults (vector_weight = 0.3, paraphrase_memory = false, n=32 entities, K∈{1,2,4,8,16}, paired bootstrap with 5000 resamples, seed=42).

Wiring (commit `31a6168`): evals/ablation.py::_make_embedder gained a `bge_large` choice that delegates to SentenceTransformerProvider("BAAI/bge-large-en-v1.5"); evals/entity_collision_sweep.py extended its `--embed` argparse choices accordingly. Three unit tests pin dim=1024 and a distinct embedding from MiniLM (tests/unit/test_make_embedder_bge.py).

A.14.1 BGE-large vs. MiniLM, paired per-query Δhit@1 (95% CIs)

Source: bench/results/ec_bge_vs_minilm_ci.json, generated by scripts/ec_bge_vs_minilm_ci.py (paired across the per-query records in the underlying sweep JSONs by query text and collision degree). Δ = BGE vector-fusion hit@1 minus ST MiniLM vector-fusion hit@1; "sig" = paired bootstrap 95% CI excludes 0.

arm	p50 (ms)	p95	p99	mean
baseline	40.94	66.17	85.94	46.23
prf	40.08	61.44	80.70	44.78
sp	40.38	62.54	85.92	45.10
both	46.85	82.18	86.09	49.02

idf_min_rarity	base h@1	gated h@1	Δh@1	±SE
None	0.2333	0.1417	−0.0917	0.0186
0.0	0.2333	0.1417	−0.0917	0.0186
0.3	0.2333	0.1417	−0.0917	0.0186
0.5	0.2333	0.1417	−0.0917	0.0186
0.7	0.2333	0.1417	−0.0917	0.0186
0.9	0.2333	0.1417	−0.0917	0.0186

Lexical-discriminator tags (`technical`, `tool`, `service`):

Intent-style tags (`project`, `preference`):

A.14.2 Verdict — bigger encoder is not uniformly better

The paired CIs reject the encoder-capacity hypothesis cleanly:

- **`technical` and `tool`** (lexical-discriminator regime, where surface-form proper-noun retrieval dominates): BGE *significantly loses* at every K∈{8,16} on `technical` and at every K∈{4,8,16} on `tool`. The `technical` K=8 deficit is the largest cell in the panel: −11.7pp, [−16.4, −7.0].
- **`project`** (intent-style, where MiniLM was already weakest): BGE *significantly wins* at every K∈{2,4,8,16}. The K=8 lift +14.4pp [+9.4, +19.5] is exactly the symmetric mirror of the `technical` loss.
- **`service` K=16** is the only lexical-tag cell where BGE wins, and the lift is small (+3.1pp) and right at the CI boundary. `preference` is null at every K.

A parsimonious mechanistic reading: BGE's contrastive pretraining emphasises semantic paraphrase and de-emphasises surface-form lexical discriminators; on a closed-vocabulary, proper-noun-answer regime this is a liability, not an asset. We do not claim a causal proof here — the point is the falsification: encoder size alone is not the binding constraint. The §4.3 two-axis interpretation **survives a 2.7×-parameter encoder swap, and in fact strengthens** — bigger model shifts the lift along the two axes rather than uniformly raising it.

Operational implication for v0.2 defaults. The v0.2 ship is MiniLM-384 with vector_weight = 0.3. We retain MiniLM as the default: BGE's lexical-tag deficit and intent-tag surplus roughly offset across the five-tag mean, BGE costs ~3× MiniLM per-query latency, and a workload-targeted embedder swap is exactly the kind of decision the §4.3 two-axis interpretation is designed to inform — not a property of the default.

Artifacts: bench/results/ec_bge_large_{service, preference,project,technical,tool}_n32_K16{,_ci }.json, bench/results/ec_bge_vs_minilm_ci.json, scripts/run_bge_sweeps.sh (driver), scripts/ ec_bge_vs_minilm_ci.py (paired CI generator), tests/unit/test_make_embedder_bge.py.

A.14.3 BGE-large on natural data — does the synthetic lexical/intent split replicate?

Question. A.4.16.1–.2 falsified the encoder-capacity hypothesis on a *synthetic* entity-collision protocol. A reviewer can reasonably ask whether the same finding survives on a real benchmark, where natural language mixes lexical and intent regimes within every query and the type taxonomy is dataset-defined rather than tag-controlled. We re-ran the LongMemEval-S baseline arm under BAAI/bge-large-en-v1.5 on the **full n=500 LongMemEval-S panel** and paired per-`question_id` against the §4.5 full-500 default-embedder baseline (bench/results/lme_full500_k10_baseline.json).
All 500 BGE question_ids matched a default-baseline record exactly; the comparison is fully paired across all six question_type cells.

Source: bench/results/lme_n500_

arm	overall h@1	Δh@1 vs baseline	CI 95%	pref-slice h@1
baseline	0.8100	—	—	0.3667
prf (raw, gate=None)	0.7700	−0.0400	[−0.0620, −0.0180]	0.4333
prf + anchor_share_max=0.7	0.7700	−0.0400	[−0.0620, −0.0180]	0.4333
prf + anchor_share_max=0.5	0.7700	−0.0400	[−0.0620, −0.0180]	0.4333
prf + anchor_share_max=0.4	0.7700	−0.0400	[−0.0620, −0.0180]	0.4333

tag	K	n_paired	ST hit@1	BGE hit@1	Δ (BGE−ST)	95% paired CI	sig
technical	4	128	0.383	0.352	−0.031	[−0.086, +0.023]	
technical	8	256	0.316	0.199	**−0.117**	[−0.164, −0.070]	
technical	16	512	0.168	0.105	**−0.062**	[−0.090, −0.035]	
tool	4	128	0.391	0.320	**−0.070**	[−0.125, −0.016]	
tool	8	256	0.242	0.180	**−0.062**	[−0.102, −0.023]	
tool	16	512	0.131	0.103	**−0.027**	[−0.047, −0.006]	
service	4	128	0.328	0.367	+0.039	[−0.016, +0.094]	
service	8	256	0.227	0.266	+0.039	[−0.004, +0.082]	
service	16	512	0.166	0.197	+0.031	[+0.006, +0.057]	

bge_large_baseline.json (BGE arm), matched per-`question_id` against the §4.5 default-baseline file via scripts/lme_bge_vs_minilm_n500_paired_ci.py (10 000-resample paired bootstrap, seed=42, output bench/results/lme_n500_bge_vs_default_ci.json).

Verdict — significant headline lift, with a structured per-type pattern. The full-panel paired CI lands well clear of zero on overall hit@1 (+5.8 pp [+3.2, +8.6]) and overall hit@10 (+2.4 pp [+1.0, +4.0]). The gain decomposes structurally: it is concentrated on the question_types where the dense side has the most paraphrase work to do — multi-session (+8.3 pp hit@1), temporal-reasoning (+6.8 pp hit@1), single-session-assistant (+7.1 pp hit@1) — and is null on the cells where lexical overlap already dominates retrieval (single-session-user, knowledge-update). The single-session-preference cell — the dominant residual error mode in §4.5 (default hit@1 = 0.367) — moves +16.7 pp on hit@1 (CI floor at exactly 0, n=30 underpowered) and is significant on hit@10 (+13.3 pp [+3.3, +26.7]).

This **reverses the n=100 preliminary** reported in earlier drafts of this subsection, where every paired CI touched zero. The n=100 result was a power story, not a real null: at n=100, sampling only the single-session-user (70) and multi-session (30) cells, the cells where BGE actually pays — temporal-reasoning, single-session-assistant, single-session-preference — were entirely outside the panel, and the multi-session signal (+8.3 pp at n=500) sat just under the n=30 noise floor in the preliminary. We retain the synthetic-data §A.4.16.2 verdict (entity-collision tag-conditional, not headline) as the *synthetic* finding, but on real LongMemEval-S the encoder upgrade *does* move the headline.

The §A.4.16.2 tag-conditional pattern (technical/tool regress, `project` improves) does not appear here in opposite-sign form — every significant cell in the natural-data panel moves *up*. The synthetic regression cells map to a tag taxonomy LongMemEval does not directly expose, so we flag this as a non-mapping rather than a contradiction: on the question_types LongMemEval enumerates, the dense-side gain is either positive or null, never negative.

Latency cost. BGE-large at n=500 spent ≈2 h 57 min wall on M4 Pro MPS (ENGRAM_ST_DEVICE=mps, ENGRAM_ST_BATCH=256, fp32) — 21.5 s/instance ingest, vs the default hashtrigram-256 baseline at ≈4 m 25 s (524 ms/instance ingest) and the prior n=100 CPU run at 150.8 s/instance (a 7× MPS speedup over CPU). Per-query recall is still ≈11× slower than the default. The headline gain is real and significant; the operational ratio for a default-flip is still unfavorable on commodity CPU, but on accelerator hardware (MPS / CUDA) BGE-large becomes a defensible

tag	K	n_paired	ST hit@1	BGE hit@1	Δ (BGE−ST)	95% paired CI	sig
project	2	64	0.500	0.609	**+0.109**	[+0.031, +0.203]	
project	4	128	0.312	0.453	**+0.141**	[+0.070, +0.211]	
project	8	256	0.164	0.309	**+0.144**	[+0.094, +0.195]	
project	16	512	0.082	0.166	**+0.084**	[+0.051, +0.115]	
preference	2	64	0.609	0.625	+0.016	[−0.047, +0.078]	
preference	4	128	0.430	0.445	+0.016	[−0.047, +0.078]	
preference	8	256	0.289	0.266	−0.023	[−0.078, +0.027]	
preference	16	512	0.184	0.199	+0.016	[−0.018, +0.049]	

cell	n	default hit@1	BGE hit@1	Δ (BGE − default)	95% paired CI	sig
overall	500	0.810	0.868	**+0.058**	[+0.032, +0.086]	
single-session-user	70	0.914	0.900	−0.014	[−0.071, +0.029]	
single-session-assistant	56	0.821	0.893	**+0.071**	[+0.018, +0.143]	
single-session-preference	30	0.367	0.533	+0.167	[0.000, +0.333]	
multi-session	133	0.805	0.887	**+0.083**	[+0.023, +0.143]	
temporal-reasoning	133	0.812	0.880	**+0.068**	[+0.015, +0.120]	
knowledge-update	78	0.885	0.897	+0.013	[0.000, +0.038]	

workload-targeted upgrade rather than a falsified hypothesis.

Default-embedder decision. The v0.2 ship default remains MiniLM-384 + vector_weight = 0.3 on the basis of (i) per-query and per-ingest latency on commodity CPU hosts (the v0.2 deployment target), (ii) the §4.3 axis-1/axis-2 framing — fusion weight is the binding control, encoder capacity is a secondary lever — and (iii) reproducibility/portability of the default config. The +5.8 pp hit@1 / +2.4 pp hit@10 BGE result is documented here as a **workload-targeted upgrade path**, not as a recommendation for the default. Operators with accelerator hardware and an a-priori workload mix concentrated on multi-session / temporal-reasoning / single-session-assistant queries should expect a real lift from the swap; users on CPU hosts with mixed workloads should not. The §4.3 two-axis interpretation thus survives intact: encoder capacity is a real second axis, but it is the second axis, not the first.

Artifacts: bench/results/lme_n500_bge_large_baseline.json (BGE arm n=500 raw), bench/results/lme_n500_bge_vs_default_ci.json (paired CI), scripts/lme_bge_vs_minilm_n500_paired_ci.py (CI generator). The earlier n=100 preliminary — bench/results/lme_n100_k10_baseline_bge.json and bench/results/lme_n100_bge_vs_default_ci.json — is retained on disk for audit but superseded by the n=500 panel.

A.14.4 RM3 baseline arm (AUDIT-D) — PRF cannot substitute for a dense encoder

Question. §5.1 already rejects *heuristic* top-k query expansion as a recovery mechanism for the BM25-only intent-tag null. A reviewer might reasonably ask whether a *scored* relevance-model expansion in the Lavrenko-Croft family — RM3 (Lavrenko and Croft, 2001), the canonical IR baseline — would change the verdict. RM3 mixes a learned expansion-term distribution against the original query terms via a λ interpolation, weighting expansion terms by their mean P(term | top-k documents) under a uniform document prior. We ran RM3 across the entity-collision grid, BEIR FiQA, and LongMemEval-S n=500 with Anserini-default hyperparameters (top-k=10, num-terms=10, λ=0.5, ε=0.01) to test whether the additional scoring machinery rescues PRF.

Implementation. evals/rm3.py implements RM1 (uniform doc prior) plus RM3 mixture in pure Python (no `src/engram/` modification, no GPU dependency). The two-pass dance — BM25(q) → expand → BM25(q′) — runs externally to Engram core; the BEIR adapter, the LongMemEval adapter, and the entity-collision sweep each gain an RM3 arm via dedicated CLI flags (`--arm rm3`, `--rm3`, `--rm3` respectively). 7 unit tests (tests/unit/test_rm3.py) cover tokenization, empty input, high-IDF term selection, expanded-query string rendering, top-k clamping, stopword filtering, and missing doc text. Test count grew

cell	n	default hit@10	BGE hit@10	Δ (BGE − default)	95% paired CI	sig
overall	500	0.932	0.956	**+0.024**	[+0.010, +0.040]	
single-session-user	70	0.957	0.957	0.000	[−0.043, +0.043]	
single-session-assistant	56	0.911	0.929	+0.018	[0.000, +0.054]	
single-session-preference	30	0.833	0.967	**+0.133**	[+0.033, +0.267]	
multi-session	133	0.962	0.977	+0.015	[−0.015, +0.045]	
temporal-reasoning	133	0.925	0.962	**+0.038**	[+0.008, +0.075]	
knowledge-update	78	0.923	0.923	0.000	[0.000, 0.000]	

250 → 257; no regressions.

Headline result — paired CIs vs BM25 baseline.

LongMemEval-S n=500 paired by `question_id`, B=10000 bootstrap resamples, seed=42 (scripts/lme_rm3_vs_bm25_n500_paired_ci.py, bench/results/lme_n500_rm3_vs_bm25_ci.json):

Verdict — three findings, all sharpening rather than weakening upstream claims.

First, the single-session-preference cliff is RM3-invariant. Δhit@1 = +0.000 *exactly* on all 30 paired instances — RM3 returns the same wrong session as BM25 in every case. The CI is degenerate because every paired delta is zero. This confirms that the §A.4.16.3 intent-tag weakness is a structural property of the lexical retrieval channel, not a BM25-specific artifact that scored expansion could fix. **§4.6 / §5.1 conclusion strengthens.**

Second, RM3 SIG-regresses single-session-user Δhit@1 by 7.1 pp (CI [−14.3, −1.4] excludes zero). The same query-drift failure mode the entity-collision `technical` cell exposes at low K (−87.5 pp Δhit@1 vs BM25 at K=1, see entity-collision panel below): when the BM25 first-pass already pinpoints the right doc, expansion terms mined from the top-k (which include adjacent-but-wrong sessions) pull the right answer down rather than up. This is the textbook Lavrenko-Croft 2001 failure regime — high-quality first-pass retrieval makes RM3 strictly worse than BM25 alone.

Third, the recall-broadening behavior the IR literature reports for RM3 is corpus-dependent. On BEIR FiQA (n=648, 57,638-doc corpus, financial conversational queries), recall@100 lifts +3.5 pp vs BM25-only (0.5079 vs 0.4725) at the cost of −1.6 pp ndcg@10 — the classic precision/recall trade-off Lavrenko-Croft 2001 documents. On LongMemEval-S, the trade-off inverts: hit@k regresses −1.4 pp SIG (CI [−2.6, −0.4]) because the BM25 first-pass already has the right session in top-10 most of the time, and expansion adds noise without adding coverage. The published RM3 wins are on retrieval workloads where BM25 *under-recalls*; agent-memory haystacks (several-hundred-session conversational logs) sit in the regime where BM25 *already-recalls* and RM3 cannot help.

Entity-collision under RM3 — refining the lexical-vs-intent axis.

Per-tag Δhit@1 vs BM25-only at the same n_entities=32, K=1..16, seed=42 protocol that drives §4.2 (bench/results/ec_rm3_*_n32_K16.json):

Two structural patterns. (i) RM3 *helps* on `service` at K=2 (+4.7 pp), where the expansion terms — concrete service nouns from the top-k docs — are diagnostic of the right answer; (ii) RM3 *catastrophically drifts* on `technical` at low K (−87.5 pp at K=1, −43.8 pp at K=2), where the open-vocabulary technical jargon in the top-k produces expansion terms that are confused with adjacent-but-wrong technical content. The lexical-vs-intent dichotomy of §4.3 is too coarse for RM3: the per-tag failure factors further by whether expansion terms are diagnostic (closed-vocab service nouns) or distracting (open-vocab technical jargon).

Comparison to the §3.1.1 latency-cost table. RM3 occupies a fourth operating point: zero model-load like HashTrigram-256 (no embedder, FTS5-only), ~1.5 ms/doc CPU ingest (faster than BGE-large by three orders of magnitude on CPU), ~259 ms query latency on FiQA because of the two-pass BM25 dance. Neither the HashTrigram-256 recovery pattern (CI-positive on `service` K=16, `tool` K∈{4,8,16}) nor the MiniLM-384 universal lift replicates under RM3. **RM3 is strictly dominated** by MiniLM-384 on every cell where MiniLM is CI-positive, and by BM25 alone on the lexical-anchor cells where PRF expansion drifts. Reviewer-relevant scope-defense: under our hyperparameter freeze and on our corpora, RM3 is not a viable substitute for either a 256-dim hash trigram or a learned dense encoder.

Hyperparameter sensitivity disclaimer. All

question_type	n	Δhit@1 [95% CI]	Δhit@k [95% CI]
overall	500	−0.014 [−0.030, +0.000]	**−0.014 [−0.026, −0.004]** SIG
single-session-user	70	**−0.071 [−0.143, −0.014]** SIG	+0.000 [exact]
multi-session	133	−0.008 [−0.030, +0.015]	−0.015 [−0.038, +0.000]
single-session-preference	30	+0.000 [exact zero]	−0.100 [−0.233, +0.000]
temporal-reasoning	133	−0.015 [−0.053, +0.023]	−0.015 [−0.038, +0.000]
knowledge-update	78	+0.013 [+0.000, +0.039]	+0.000 [+0.000, +0.000]
single-session-assistant	56	+0.000 [exact zero]	+0.000 [+0.000, +0.000]

tag	K=1	K=2	K=4	K=8	K=16
preference	+0.000	+0.000	+0.000	+0.000	+0.000
project	+0.000	−0.063	−0.039	−0.012	+0.006
technical	**−0.875**	**−0.438**	**−0.148**	−0.035	+0.000
service	+0.000	**+0.047**	+0.016	+0.008	+0.002
tool	−0.094	+0.000	+0.008	+0.004	**+0.016**

RM3 cells use Anserini defaults. A targeted hyperparameter search (lower λ to de-emphasize expansion, lower num-terms to cap drift) might recover some single-session-user regression but cannot in principle move the single-session-preference cliff (paired Δ = +0.000 *exactly* means both arms produce identical rankings on those queries — the expansion is selecting non-discriminating terms regardless of mixture weight). We accept the Anserini defaults as the published reference point and note that an exhaustive λ sweep is queued for v0.3.

Reproducer paths. evals/rm3.py (module), tests/unit/test_rm3.py (7 unit tests), scripts/lme_rm3_vs_bm25_n500_paired_ci.py (paired CI generator), bench/results/{ec_rm3_*_n32_K16,beir_fiqa_rm3,lme_n500_rm3, lme_n500_rm3_vs_bm25_ci}.json (artifacts).

A.14.5 BEIR-3 — second natural-data anchor (BGE-large + hybrid)

Question. Does the BGE-large + hybrid configuration that wins on the synthetic entity-collision grid (§4.3) and on LongMemEval multi-session (§A.4.16.3) also produce sensible numbers on a canonical retrieval benchmark, independent of LongMemEval / LoCoMo? A "yes" rules out the trivial concern that the synthetic→natural bridge in §4.6 is an artifact of LongMemEval's specific construction.

Protocol. End-to-end run against the testbed at default config (hybrid BM25+vector, vector_weight=0.3, no reranker, no expansion, no schema-extracted entities — the BEIR adapter writes raw passage text via eng.remember()). Encoder: BGE-large-en-v1.5 (1024-d). Metrics: ndcg@10, recall@100. We pick three corpus-size points to characterize how the single-writer ingest path scales: FiQA (small, 57k), NQ (large, 2.68M), HotpotQA (very large, 5.23M).

Results (FiQA, NQ).

NQ recall@100 = 0.812 is in line with the published BGE-large single-vector range on this benchmark (no reranker, no query expansion, full corpus). FiQA's lower ndcg@10 is consistent with its narrow financial-conversational domain and BGE's lack of in-domain fine-tuning. The point is not to chase BEIR leaderboard; it is to confirm that the same config that drives §4.6 produces sensible numbers on a public canonical benchmark.

Ingest path characterization. Per-doc ingest rate measured at **39 ms/doc on FiQA** and **30.6 ms/doc on NQ** under accelerator fp32 batched-encode. The rate is clamped by per-call kernel-launch overhead on the encoder, not by the SQLite/FTS5 write path: the BEIR adapter's hot loop calls eng.remember() per doc, which calls `embed()` (single-doc) — fp16 batched encode would cut this to ≈9 ms/doc but requires an Engram.remember_batch() helper that coalesces sqlite/FTS5 writes into one transaction. That helper is queued for v0.3 and is the gating dependency for HotpotQA.

HotpotQA deferral. Projected wall-clock at the measured 30.6 ms/doc rate is ≈44 hours;

Task	n_corpus	n_queries	ndcg@10	recall@100	query p50	ingest wall
FiQA	57,638	648	0.341	0.695	277 ms	37.5 min
NQ	2,681,468	1,000	0.355	0.812	16.0 s	22.8 h

corpus-size penalty on the single-writer SQLite path pushes the realized rate higher (the 1M-ingest curve in §A.4.14 shows p99 +13% from 100k → 1M; HotpotQA is 5× larger again). We do not ship HotpotQA in v0.2 to avoid (a) anchoring a multi-day single-pass run inside the deadline window with non-negligible failure probability, and (b) shipping a result that would be re-run on a different code path (remember_batch) the moment v0.3 lands. The deferral is documented in §75 (Limitations) and queued in §B (TODO-RESEARCH.md v0.3 milestone).

Why this counts as a second natural-data anchor. BEIR FiQA and NQ source from a different distribution than LongMemEval (financial QA / encyclopedic QA vs synthesized multi-turn personal-assistant sessions) and from a different distribution than LoCoMo (long-form conversation grounded in personae). The two-axis interpretation in §4.3 — BGE wins on encyclopedic / multi-session breadth, MiniLM holds the operational default — is consistent with NQ's 0.812 recall@100 (large breadth corpus, BGE thrives) and FiQA's narrower lift (small specialist corpus, less room for capacity to help). We treat the BEIR-3 numbers as confirmatory rather than headline-driving; the headline two-axis claim continues to rest on the synthetic grid + LongMemEval n=500.

Reproducer paths. evals/beir_adapter.py (module), scripts/run_beir_bge_large.py (driver, with --engram-path / --checkpoint-every for resume), bench/results/beir_{fiqa, nq}_bge_large_hybrid.json (artifacts). The resume path (.beir_progress.json keyed on task/arm/embedder/split/n_corpus) is what enables a 22.8h NQ run to survive a workstation lid-close. HotpotQA reproducer (scripts/run_beir_bge_large.py --task hotpotqa) is wired but deferred; rerun under v0.3 batched-ingest will populate beir_hotpotqa_bge_large_hybrid.json.

A.15 Schema-lifecycle invariants — the property suite that backs §A7.4.4

Question. §A7.4.4 states the schema-lifecycle reducer as a pure event-fold and asserts a five-edge DAG plus five textual invariants. §A4.2 then claims those invariants are *enforced by Hypothesis property tests* rather than asserted by spot-check. This subsection is the audit trail for that claim — a one-page index of the property surface and the classes of bug each property would have caught had we written the reducer naively.

The §B research thread (TODO-RESEARCH.md) opened with six prose invariants for the lifecycle. Five map onto the DAG-plus-fold contract of src/engram/consolidation/schema_lifecycle.py; the sixth — *schema writes serialize against extraction writes* — is a concurrency claim about the persistence layer (src/engram/store/buffer.py already takes an exclusive `fcntl.flock` on append) and is fuzzed at the buffer level. We catalogue the gates here so a reviewer can reproduce the chain of invariants → tests → bugs-prevented without grepping the test directory.

A.15.1 Invariant ↔ test ↔ bug-class table

A.15.2 Cross-feature compositions

Two further suites cover the *interaction* of the lifecycle reducer with adjacent v0.2 features — necessary because invariants #1–#6 each hold in isolation but the production engine composes them:

- tests/property/test_dedup_lifecycle_composition_stateful.py — asserts that write-side cosine deduplication (§A7.4.2) and lifecycle emissions are mutually independent: dedup decisions are unchanged by interleaved LifecycleEvents on the same schema, and the lifecycle projection is unchanged by dedup absorption (i.e., a fact being absorbed never fires a spurious lifecycle transition). Caught a draft of §A7.4.2 that briefly considered emitting a synthetic BUMP_VERSION on dedup-merge.
- tests/property/test_extraction_conf_lifecycle_composition_stateful.py — asserts that per-fact extraction confidence (§A7.4.2) and the respect_schema_lifecycle retrieval filter factorise: the score of a candidate from a non-deprecated schema does not depend on lifecycle history, and a deprecated-schema filter is independent of the candi-

§B invariant (TODO-RESEARCH.md)	Property gate(s)	Bug-class caught
#1 *Status is monotone modulo recovery* (only the four DAG edges are legal).	tests/property/test_schema_lifecycle.py::test_lenient_reduce_respects_dag, ::test_strict_rejects_promote_from_deprecated.	A reducer that accepts deprecated → promoted directly, or rolls back promoted → inferred without a fresh-window `RECOVER`. Catches off-by-one transitions and "we forgot to enumerate this edge" omissions.
#2 *Promotion is deterministic and replayable* (pure fold over the event log).	::test_reduce_is_deterministic, ::test_initial_snapshot_equivalence, tests/property/test_lifecycle_projection_roundtrip.py::test_projection_equals_direct_reduce, ::test_projection_resumable_via_partial_replay.	Any wall-clock / RNG / external-state read snuck into the reducer. Replaying the same JSONL twice would diverge; the property would fail on the second run. Also catches "resume from offset N" bugs in the projection layer.
#3 *Promotion never invalidates stored properties* (version monotone under PROMOTE/DEPRECATE; only BUMP_VERSION increments).	::test_bump_version_preserves_status_and_counts.	A reducer that lazily bumps version on every PROMOTE — silently invalidating downstream rows tagged schema_version=v1 because the snapshot now reads v2.
#4 *Schema writes serialize against extraction writes* (the concurrency claim).	tests/property/test_lifecycle_concurrent_append.py — four sub-invariants CL-I1..CL-I4 (lossless persistence, projection consistency, per-schema causal-order legality, per-kind histogram conserved). N writers × shared threading.Barrier.	Torn JSONL frames under racing `O_APPEND`, lost events under flock failure, snapshot diverging from reduce_events(scan_order).
#5 *Demotion is reversible only through evidence* (RECOVER requires fresh `window_id`).	::test_recover_requires_fresh_window.	An oscillation bug where the same evidence window thrashes a schema between live/dead — would let a single bad window cause unbounded RECOVER/DEPRECATE pairs.
#6 *Lifecycle decisions are events, not in-place mutations* (the architectural claim).	::test_create_on_existing_strict_raises, ::test_unknown_schema_strict_raises, plus the cache-fuzz suite tests/property/test_lifecycle_snapshot_cache_fuzz.py::test_c1_random_interleave_equivalence, ::test_c2_append_only_offset_monotone_and_eof_hit, ::test_c3_rotate_increments_misses.	A reducer that mutates an existing state on a stray CREATE (silently doubling promote_count on replay), or a snapshot cache whose fast-path disagrees with a from-scratch fold. The cache-fuzz suite is what makes invariant #2 hold *under the fast path*, not just under cold replay.

date's extraction-confidence value. The factorisation property is what allowed §A.4.6's bisection to attribute the retrieval delta to *extraction*, not to lifecycle filtering.

A.15.3 Headline numbers

The full property surface (across the seven files referenced above) runs at **27 properties** (11 in the core reducer suite, 4 in the projection round-trip, 6 in the snapshot-cache fuzz, 6 in the concurrent-append harness) plus the two stateful composition suites. Wall-clock under the production Hypothesis settings is $\leq$8 s on a Ryzen-class laptop; the suite is part of every `pytest -q` run that gates a commit. 1715 passed, 3 skipped as of the cron run dated 2026-05-24 (commit `cc5f72e`).

A.15.4 Why this lives in the appendix, not §3

A natural alternative is to inline the invariant-bug-test table into §A7.4.4. We deliberately keep §A7.4.4 a prose specification of *what* the reducer is — five rules and a DAG — and push *why we trust the implementation* here. A reviewer who accepts §A7.4.4's contract on inspection can skip §A.4.17; a reviewer who wants to audit the gate between specification and implementation gets the chain of evidence in one place. This mirrors the §A7.3 / §A.4.7 split: methods state the mechanism, appendix shows the falsification budget.

Artifacts: tests/property/test_schema_lifecycle.py (163 LoC), tests/property/test_lifecycle_projection_roundtrip.py (126 LoC), tests/property/test_lifecycle_snapshot_cache_fuzz.py (161 LoC), tests/property/test_lifecycle_concurrent_append.py (253 LoC), tests/property/test_dedup_lifecycle_composition_stateful.py (318 LoC), tests/property/test_extraction_conf_lifecycle_composition_stateful.py (410 LoC), src/engram/consolidation/schema_lifecycle.py (267 LoC, the reducer under test), src/engram/consolidation/lifecycle_projection.py (405 LoC, the projection layer).

A.16 Claim → section → artifact registry

The full claim-to-artifact registry (26 result tables, 37 reproduce scripts) is verified by scripts/verify_repro_artifacts.sh against the on-disk filesystem at every release. The digest below covers headline claims a reviewer is most likely to interrogate; the full registry is in paper/REPRODUCIBILITY.md §1.

B Appendix B. Security audits of Engram retrieval and storage primitives

This appendix collects security-side threat analyses of Engram's retrieval pipeline and storage primitives. These are systems-side audits of ACL/access-control side-channels in PRF query expansion, share_prior reranking, schema-lifecycle caches, BM25/vector candidate pools, mechanical merge, FactExtraction, write-side cosine dedup, governed-memory primitives, and cross-channel coupling. They are *not* measurement-side threats to the retrieval claims of §4 — those are addressed in §6 of the main paper.

We retain them in the appendix because they document the security-audit methodology that supports the Engram artifact release; readers interested only in retrieval results can skip this appendix.

Section numbers preserve the original §6.X numbering as §A.6.X for cross-reference stability.

B.1 PRF query-expansion ACL side-channel (closed)

Pseudo-relevance-feedback (PRF, §A7.3 / §A.4.7) runs a first-pass retrieval and mines dominant entities from the top-K texts to construct an expanded query. In the original wire-up the first pass executed at the RetrievalEngine layer — strictly upstream of the outer per-result ACL filter in Engram.recall() — so the entity-mining pool could include memories the actor lacked READ scope for. The expanded query, and therefore the actor's final ranking over its *own* memories, then depended on the cross-agent corpus. This is a side-channel oracle: an actor with `scope='own'` could detect the presence of specific tokens in another agent's private memories by observing rank-perturbations on its own queries, even though no cross-agent content ever reached the user-visible output.

The repro is small: Alice owns four `notes …` memories, two with disambiguating entities (Apollo, Beta). Bob privately owns 20 docs of the form notes Apollo Apollo item N. Without the fix, PRF mines `apollo` from Bob's docs, rewrites Alice's query to notes apollo, and Alice's Apollo

Claim	Section	Artifact
Hash trigram lifts lexical tags at deep K	§4.2	ec_sweep_hash_*_n32_K16_ci.json
MiniLM dominates both axes at K≥4	§4.2	ec_sweep_st_*_n32_K16_ci.json
BGE-large does not uniformly dominate MiniLM	§4.3, §A.4.16	ec_bge_large_*_n32_K16_ci.json
LongMemEval n=500 BGE paired CI vs MiniLM (SIG)	§4.6, §A.4.16.3	lme_n500_bge_large_baseline.json, lme_n500_bge_vs_default_ci.json
RM3 cannot rescue intent-tag null (AUDIT-D)	§5.1, §A.4.16.4	lme_n500_rm3_vs_bm25_ci.json, ec_rm3_*_n32_K16.json
Single-session-preference recall cliff (intent-tag null)	§4.6	lme_full500_k10_baseline.json
Adaptive-vw on LoCoMo is null	§4.4	locomo10_st_learned_router_hit_at_1_leakfree.json
1M write-latency tail (p99 +13% from 100k → 1M)	§A.4.14	ingest_1m_*_buckets.json
share_prior rank-0 invariant (150 arms, Δhit@1 ≡ 0)	§A7.2, §A.4.7	SHARE_PRIOR_REPORT.md
BEIR-3 FiQA (BGE-large, hybrid, ndcg@10=0.341, recall@100=0.695)	§4.6, §A.4.16.5	beir_fiqa_bge_large_hybrid.json
BEIR-3 NQ (BGE-large, hybrid, ndcg@10=0.355, recall@100=0.812, 2.68M docs)	§4.6, §A.4.16.5	beir_nq_bge_large_hybrid.json

doc rises one rank. Flip Bob's corpus to Beta and Alice's Beta doc rises instead. The ACL *recall* filter still strips Bob's results from the output — the leak is purely in Alice's own ranking.

Closure (commit `07d5c35`, branch feat/prf-acl-side-channel-fix). RetrievalEngine.search() accepts an optional _acl_filter: (Memory) -> bool that _search_with_prf applies to the first-pass results *before* the PRF expander is called. Engram.recall() binds the filter to self._acl_allows_read(actor, mem.agent_id) for the current actor. The filter is inert when ACL is disabled, and federated actors with `scope='*'` (e.g. an auditor / reviewer grant) still mine the full pool — the escape hatch is preserved by the same permission model that governs federated reads. Pinned by tests/adversarial/test_prf_acl_side_channel.py (7 cases covering rank invariance across four Bob corpora, expander pool isolation, the federated escape hatch, and ACL-disabled no-regression). Pre-fix: 4 of the 7 cases fail with diagnostic diffs; post-fix: 7/7 pass. Full suite remains green at 1581 passing.

B.2 PRF IDF-rarity ACL side-channel — closed

The PRF expander has a second cross-agent signal beyond the mining pool addressed in §A.6.6: the §A.4.15g *IDF-rarity gate* drops candidate entities whose corpus rarity falls below a threshold (default idf_min_rarity = 0.5). Pre-fix, the rarity score was $1 - \texttt{df/N}$ where both df and N were computed against the **global** FTS index — including memories the actor cannot READ. The keep/drop decision for an Alice-pool entity therefore depended on Bob's private corpus.

The leak is detectable end-to-end in numbers, not just in principle. Alice owns four `notes …` memories; Apollo appears in one of them (df=1 in Alice's slice, N=4 rarity=0.75, gate keeps Apollo). Bob privately writes 50 notes Apollo Apollo … memories. Pre-fix rarity collapses to df/N ≈ 51/54, giving rarity ≈ 0.06 — below 0.5 — and the gate **drops** Apollo from Alice's PRF expansion, silently flipping her ranking on her own corpus. Switching Bob's corpus to Beta-dense flips the gated entity from one to the other: the post-expansion ranks vary with Bob's vocabulary even though no cross-agent content reaches Alice's output. This is the classic shape of a side-channel oracle.

Closure (commit `e0aa422`). RetrievalEngine._build_prf_rarity_lookup gains an allowed_agents: set[str] | None parameter that scopes **both** the df numerator and the N denominator via m.agent_id IN (?, …). Engram._prf_rarity_allowed_agents(actor) computes the allow-list from the ACL grant for the current actor: `None` when ACL is disabled, when the actor has `scope='*'`, or when the actor holds Permission.FEDERATED (preserving the federated escape hatch); `{actor, ''}` for `scope='own'`, mirroring Grant.can_access. `recall()` threads it through the new _rarity_allowed_agents private kwarg on `search()`. Inert in all single-actor and federated configurations.

Pinned by tests/adversarial/test_prf_idf_acl_side_channel.py (7 properties): four rank-invariance arms under the IDF gate × Bob signal, a direct rarity-lookup numeric assertion (rarity(Apollo) ≥ 0.5 post-fix vs. ≈ 0.029 pre-fix in the same corpus), the federated reader keeping global df, and ACL-disabled no-regression. Pre-fix: 3 of the 7 cases fail (the direct lookup canaries); the behavioural rank tests do not perturb in this corpus geometry because the expander-pool fix from §A.6.6 already strips Bob's docs from the entity-mining input — but the gate's numeric decision is still demonstrably wrong, and an adversary with control over Alice's pool composition could still pivot the gate. Post-fix: 7/7 pass. Full suite: 1581 → 1588 passing, 3 skipped, 181 deselected, 226 s.

B.3 share_prior reranker ACL side-channel (§D-share-prior-acl)

A third sibling of the §A.6.6 / §A.6.7 leaks: the §96 *share_prior* reranker (src/engram/retrieval/rerankers/share_prior.py) builds an undirected entity-sharing graph over the post-fusion candidate pool and adds a bounded $\alpha \cdot$ deg / max_deg boost to each candidate's score. The reranker runs at the RetrievalEngine layer, which sits *upstream* of Engram.recall()'s outer ACL filter. Without an ACL-aware filter on the reranker pool, the entity-sharing graph spans cross-agent docs, so each Alice doc's `degrees[i]` counts edges into Bob's private corpus and the `max_deg` normaliser is global. The visible output is still all Alice's (the outer filter strips cross-agent rows), but their *ranking* — and even their absolute scores — are now a function of Bob's private content.

Closure: RetrievalEngine.search() now drops cross-agent docs from `results` *before* slicing the rerank pool, gated on the same `_acl_filter` already threaded through for §A.6.6. None preserves federated / single-actor behaviour (no over-correction). Inert when no reranker is configured.

Pinned by tests/adversarial/test_share_prior_acl_side_channel.py (8 tests / properties): five rank-and-score-invariance arms under diversifying / dense / orthogonal Bob corpora, a federated `scope='*'` reviewer who must still see Bob's docs, the ACL-off no-regression path, and a direct reranker-pool isolation canary (spy on apply_reranker's pool, assert no `Zorbax` from Bob). Pre-fix: the canary fails immediately (15 Zorbax docs in the pool); the rank-invariance arms happen to hold because share_prior's rank-0 preservation cap absorbs the boost differences in this corpus geometry — but the leak is unambiguous at the pool layer and would surface as score perturbation on a slightly larger α or denser sharing graph. Post-fix: 8/8 pass. Full suite: 1588 → 1596 passing, 3 skipped, 181 deselected, 226 s.

B.4 Schema-lifecycle cache ACL audit — clean

The §A7.4.4 schema-lifecycle gate replays the buffer's CONSOLIDATION_SCHEMA_LIFECYCLE event stream through a mtime-keyed CachedLifecycleSnapshot and uses the resulting {schema_id: SchemaState} map to drop DEPRECATED MemoryType.SCHEMA candidates from a recall's result set. Three siblings of §A.6.6–§A.6.8 (PRF×ACL, PRF IDF-rarity, share_prior reranker) all turned out to be silent cross-agent ranking channels; we audited the lifecycle cache for the same shape of leak.

Audit closure (3 invariants + 1 positive control, pinned by tests/adversarial/test_lifecycle_cache_acl_side_channel.py):

- **ACL-LC-1:** Alice's recall ranking over FACT memories is bit-identical (content + score equality) under three Bob-emitted lifecycle traffic patterns: empty, single CREATE+DEPRECATE on bob_schema_x, and a six-event churn (CREATE/PROMOTE/DEPRECATE/BUMP_VERSION across three Bob schema ids). The lifecycle filter only fires on candidates of type SCHEMA, so cross-agent FACT ranking cannot depend on Bob's schema status. Pinned for all three arms.
- **ACL-LC-2:** When a SCHEMA candidate's `schema_id` collides with a DEPRECATED entry in the snapshot, it is suppressed *regardless of which actor emitted the lifecycle event*. This is the **intended** behaviour — schemas are `agent_id=''` (system-wide patterns), and the lifecycle DAG is global by design — but we pin it explicitly so any future "scope lifecycle by emitter" change has to refresh this test rather than silently change semantics.
- **ACL-LC-3:** With respect_schema_lifecycle=False, lifecycle traffic is fully inert — even a CREATE+DEPRECATE pair on a present SCHEMA leaves it reachable. Confirms the cached snapshot is not silently feeding any other recall signal (no second consumer).
- **Positive control:** With the gate on, an explicit DEPRECATE *does* suppress a present SCHEMA. Paired with ACL-LC-3, the two show the cache is reachable when on and inert when off, so ACL-LC-3 is a real null and not a silent skip.

Result: no leak. The lifecycle cache passes the cross-agent audit without code changes. Closes the last open NEXT-list audit item ("confirm no other recall paths feed cross-agent texts into a learned signal"). Full suite: 1596 → 1602 passing, 3 skipped, 181 deselected, 226 s.

B.5 Cross-actor schema-id targeted suppression — pinned threat

§A.6.9 named, in passing, that DEPRECATE on a colliding `schema_id` is "global by design." Pinned here as an explicit, named threat-model statement instead of a quiet implementation detail.

Attack. A malicious actor that learns a victim schema's `schema_id` can append CREATE+DEPRECATE lifecycle events on that id and silently suppress the schema from every other actor's recall. The lifecycle DAG is intentionally global (schemas are `agent_id=''`, system-wide patterns), so the suppression applies to everyone uniformly. There is no per-emitter ACL on lifecycle events.

Observability. Schema ids are exposed through normal recall: any actor whose query surfaces a schema candidate sees its `memory.id` in the result row. Schema ids are mem-sc-<uuid4().hex[:12]> — 48 bits of entropy, so blind guessing costs ~2^47 writes per landed suppression on average, but observed ids are free.

Pinned tests (tests/adversarial/test_schema_id_targeted_suppression.py, +7 tests):

- **SC-ID-1:** A CREATE+DEPRECATE pair on a known schema id globally suppresses the schema. Worst-case attack surface, pinned.
- **SC-ID-2:** A DEPRECATE on a non-existent / mis-guessed id is a full no-op (5 parametric arms over different guess shapes). Confirms the entropy floor — guessing alone is infeasible.
- **SC-ID-3:** End-to-end chain. An actor recalls a system-wide schema, observes its id from result.memory.id, then writes CREATE+DEPRECATE; subsequent recalls miss the schema. This is the realistic attack path.

Mitigation surface (deferred — not implemented in v0.2):

- **Scope lifecycle by emitter (per-agent DAGs).** Cleanest fix; but it breaks the intended global-schema semantics, so it would need a parallel "shared-pattern" type and an explicit promotion pathway from per-agent → shared.
- **Quorum gate on DEPRECATE.** Require ≥k independent emitters or an audit-trail signature before a deprecation takes effect. Adds latency to legitimate deprecations.
- **Redact schema ids in cross-actor recall result rows.** Closes observability but breaks debuggability and any actor's ability to reference its own schemas by id.

We document this as a known, accepted limitation of the global lifecycle DAG. The system is single-tenant by intent; multi-tenant deployments must either trust all actors with respect to schema deprecation or pick one of the mitigations above. Future work item.

B.6 BM25/vector candidate-pool ACL side-channel (closed)

A fifth sibling of the §A.6.6–§A.6.9 channels — and the most direct one, because it fires on the *primary* candidate-generation path rather than on an opt-in retrieval feature. The retrieval engine builds the hybrid candidate pool by calling store.search_text(query, limit=k*5) for BM25 and vector.search(query_vec, limit=k*5) for sqlite-vec. Neither call was ACL-scoped: both scanned all agents' memories, then each candidate received a `bm25_rank` (and `vector_rank`) reflecting its position in the *global* pool. The outer ACL filter at engine.recall() (engine.py:408) stripped cross-agent docs *after* RRF fusion, so the visible output contained only Alice's docs but their fused scores — and therefore their relative order — depended on where Bob's private docs landed in the global pool.

This is a presence oracle that fires even when the reranker, PRF, and lifecycle gate are all off. In the worst case (Bob's private corpus saturates the top-`k*5` BM25 hits), Alice's recall returned the empty set despite her own corpus matching the query: a denial-of-recall as a function of Bob's content.

Closure (engine.py:_search_with_prf): drop cross-agent candidates from the pool *before* RRF fusion, then re-number `bm25_rank` and `vector_rank` so they are contiguous over the actor-visible candidates. We reuse the `_acl_filter` callable already threaded through for §A.6.6–§A.6.8. None preserves federated / single-actor behaviour exactly. Test pin: tests/adversarial/test_vector_channel_acl_side_channel.py — 3 BM25 parametric arms (orthogonal / Apollo-overlap / Beta-overlap) verify Alice's order *and* fused scores are bit-identical with and without Bob's corpus, plus a positive control that confirms the seed actually does perturb retrieval when ACL is off (so the invariance is real, not a harness

artifact).

Residual. FTS5's BM25 score itself is computed over global corpus statistics (avgdl, df, N). Re-numbering closes the *rank-position* channel, which is the only BM25-derived signal that enters fused scoring (RRF uses the rank, not the raw BM25 score). The score-magnitude channel does not enter recall scoring at all in v0.2 and so is not exploitable through the public API. We note this explicitly so future work that begins consuming raw BM25 scores re-opens the audit.

B.7 MechanicalMerge ACL side-channel (closed)

The Stage 12 MechanicalMerge consolidator is the write-side analogue of the §A.6.11 read-side channel. It iterates store.all_active() (global, no ACL scope), embeds each memory, asks the vector store for near-duplicates above its cosine threshold (default 0.95 in production, 0.90 in this test), and unconditionally moves the lower-salience side of each matching pair into MemoryState.SUPPRESSED.

The pre-fix implementation never consulted `agent_id` on the matched pair, which fused two distinct vulnerabilities into one stage:

1. **Existence oracle.** When Bob writes content semantically near-identical to Alice's, mechanical merge silently moves Bob's row into SUPPRESSED. Alice cannot read Bob's row directly under ACL — but she can observe a *state transition* on it via lifecycle metadata (memories_suppressed in the consolidation report, `state` column queries on the audit path). This separates "Bob never wrote this" from "Bob wrote a near-duplicate of my content," which the threat model forbids.

2. **Cross-tenant denial-of-recall.** In the asymmetric case where one tenant runs at higher salience than another, the louder tenant's writes systematically suppress the quieter tenant's near-duplicates. Multi-tenant deployments cannot tolerate this: a single noisy tenant could erase a quiet tenant's memories simply by writing similar content at higher salience.

Closure (pipeline.py:MechanicalMerge.run): over-fetch the candidate pool from `limit=5` to `limit=20` and skip any pair whose `agent_id` strings differ. System-owned memories (`agent_id=''`, e.g. SCHEMA prototypes) remain mergeable globally — that is the intended behaviour for shared system patterns and matches the §A.6.9 lifecycle-cache audit's treatment of schemas as agent-less.

Test pin: tests/adversarial/test_mechanical_merge_acl_side_channel.py (5 invariants).

- **MM-ACL-1** Cross-agent near-duplicates are *never* suppressed, in either salience-ordering direction (two parametric arms: Alice high / Bob low, and the symmetric reversal).
- **MM-ACL-2** Same-agent near-duplicates *are* still suppressed. This is a positive control — the fix is a scope filter, not a stage disable.
- **MM-ACL-3.** System memories with empty `agent_id` remain globally mergeable.
- **MM-ACL-4** A system memory does *not* suppress an agent-owned near-duplicate — `agent_id=''` is not a wildcard owner.

Pre-fix, three of five tests fail (MM-ACL-1 both arms + MM-ACL-4), confirming the channel is real and the patch is the minimum scope filter that closes it without breaking the same-agent merge path.

Residual. The merge stage's candidate fan-out is now bounded by `limit=20` over actor-visible duplicates. In a pathological corpus where one agent has $\geq$20 near-duplicates within the threshold, the 21st-onwards may go unmerged in a single pass; the next consolidation cycle catches them. We accept this — the alternative (full per-agent scan) is $O(n^2)$ in agent-owned content, and 20 is two orders of magnitude above the empirical near-duplicate density we see in the §4 corpora.

B.8 FactExtraction ACL inheritance bug (closed)

The most direct ACL leak in the v0.2 surface, and the worst one to have shipped — though it was caught before any external user was exposed. The Stage 4c FactExtraction consolidator distils EPISODE memories into FACT rows via an LLM call. The synthesised CONSOLIDATION_EXTRACT event carries no actor context, so the default Memory.agent_id for the resulting fact was `''` (empty — "system memory" in the v0.2 ACL model).

Pre-fix flow:

1. Alice (agent_id='alice') writes "I am meeting Mallory at 3pm at the Whitebridge."

2. Consolidation extracts the fact "Alice meets Mallory at the Whitebridge" — distilled, structured, often more searchable than the source.
3. The fact is stored with `agent_id=''`. Under Grant.can_access, *any* actor's grant matches `agent_id=''` because system-shared content is intentionally readable to all.
4. Bob's recall("Mallory") surfaces the distilled fact. Alice's ACL does not protect her — her own consolidation pipeline promoted her content into the system pool.

This is strictly worse than §A.6.11 / §A.6.12: those leak a *signal* (rank position, suppression state) about Alice's content. This one leaks the **distilled content itself**, including any facts the LLM extracted with structured `properties` — which in the dual-extraction schema (Governed Memory paper) include explicit entity bindings.

Closure (pipeline.py:FactExtraction.run): one line — fact_memory.agent_id = memory.agent_id. The source episode's owner is the only correct attribution; the consolidation event has no actor context to fall back on. Schemas keep `agent_id=''` intentionally (system-shared patterns, matching the §A.6.9 lifecycle audit).

Test pin: tests/adversarial/test_fact_extraction_acl_inheritance.py (4 invariants):

- **FE-ACL-1** Facts from Alice's episodes inherit agent_id='alice'.
- **FE-ACL-2** Facts from Bob's episodes inherit agent_id='bob'.
- **FE-ACL-3** Facts from system episodes stay `agent_id=''`. The fix forbids silent *promotion* of agent-owned content into the system pool, not legitimate system-to-system extraction.
- **FE-ACL-4** Mixed batch (Alice + Bob episodes in one stage invocation) → per-fact attribution is correct, no cross-contamination.

This is the third write-side ACL bug found in the audit pass (§A.6.12 mechanical merge, §A.6.13 fact inheritance, plus the Stage 9 schema extraction explicitly retains `agent_id=''` by design). The pattern is consistent: any stage that synthesises a new event without an actor context defaults the resulting memory to system scope, and any Memory.from_event call site needs an explicit attribution rule. Future stages must be audited against this invariant before merge.

B.9 Write-side cosine dedup ACL side-channel (closed)

Threat (§D-write-dedup-acl): the Governed Memory write-side cosine dedup (§A7.4, threshold 0.92) calls vector_store.search() over the *global* vector index. A writer Bob whose content embedding sits within the dedup threshold of one of Alice's stored memories has his write silently suppressed: engine.remember() returns the deduped event id, the projection is not inserted, and the audit log records remember_deduped status=skipped.

This produces three observable presence oracles for Bob:

1. **Audit channel:** remember_deduped fires only when a near-cosine neighbour exists *somewhere* in the store. Mining the audit log reveals which probes intersect Alice's content.
2. **Recall asymmetry:** Bob's `remember()` returns a non-empty event id, but his subsequent `recall()` over his own scope returns 0 hits. The gap is observable without any access to Alice's data.
3. **Storage delta:** the JSONL event buffer grows but Bob's projection-row count does not. Any monitor watching the event-log/projection delta sees the leak.

This is *write-side* and survives every read-side ACL fix landed through §A.6.13 — the previously-closed channels (PRF mining pool, IDF-rarity, share_prior reranker, lifecycle cache, BM25/vector candidate pool, mechanical merge, fact-extraction inheritance) all operate after the writer's content has either landed or been deduped.

Closure (store/memory.py:upsert, engine.py: remember): an `acl_filter` callable plumbed into the dedup loop. When ACL is enabled, the writer's _acl_allows_read is consulted on each candidate neighbour's `agent_id` before the cosine comparison, and candidates outside the writer's visible scope are skipped. The candidate pool is widened from K=5 → K=32 when filtering is active so top-K cross-agent neighbours don't crowd out a same-agent duplicate. Behaviour with ACL disabled is identical to the prior implementation (global dedup), preserving the legacy single-actor semantics.

Test pin: tests/adversarial/test_write_dedup_acl_side_channel.py (3 invariants):

- **WD-1** Bob writes content matching Alice's memory under ACL → Bob's projection lands and is recallable in his own scope. The failing pre-fix observation is exactly the leak.
- **WD-2** ACL disabled → cross-actor dedup still fires (regression guard for legacy single-tenant deployments).
- **WD-3** Same agent writes the same content twice → dedup fires within the writer's own scope (the legitimate behaviour we preserve; only cross-agent dedup is the leak).

Race extension (write-write contention). The §A.6.14 fix is verified under single-threaded conditions; we re-test it under contention because two `Engram` instances on the same store hold *separate* `_dedup_locks`, so ACL filtering must be correct in the absence of cross-instance serialisation. The order in Engine.remember() is store.upsert() → vector.upsert(), so a vector becomes visible to other writers only after its row is committed; this happens to make the race window *safer* than the sequential case for the leak direction, but two hazards still need pinning:

- **R1 (stale-row hazard).** A vector hit may resolve to cand is None if the row is not yet visible to the second writer's connection. The current code path treats this as "skip neighbour, do not suppress" — the safe direction. A future refactor that flipped the None-policy to "suppress" would silently reopen §A.6.14 whenever a row materialised after its vector index entry. We pin the policy with a monkey-patched store.get → None test that asserts Bob's write still lands.

- **R2 (Alice×Bob identical-payload storm).** 20-thread ThreadPoolExecutor (10 alice + 10 bob) all writing the same payload behind a `Barrier`. Invariant: each actor retains $\geq$1 row (cross-actor cannot suppress) AND same-actor dedup still bounds each actor's count to $\leq$50% of writes (regression bound — naive code with no dedup serialisation would let all 10 land per actor).

- **R3 (presence-oracle under contention).** After 5+5 concurrent same-payload writes, Bob's own-scope `recall()` must return his payload regardless of Alice's interleaving. Verified to *fail* when ACL filter is forcibly disabled — the test is real, not vacuous.

Test pin: tests/adversarial/test_write_dedup_acl_race.py (3 invariants R1/R2/R3, marker = `concurrency`).

This is the *fourth* write-side ACL bug found in the audit pass (§A.6.12, §A.6.13, §A.6.14, plus the design-intentional schema sharing). The pattern continues to be consistent: any stage that consults global state (vector index, mechanical-merge cluster, fact-extraction attribution) without an explicit ACL gate becomes a side-channel. After §A.6.14 the v0.2 audit set is: read-side closed (§§A.6.6–A.6.11), write-side closed (§§A.6.12–A.6.14), with §A.6.10 (cross-actor schema-id targeted suppression) pinned as accepted behaviour of the global lifecycle DAG.

B.10 Governed-Memory `extraction_confidence` ACL audit — clean

Threat (§D-extraction-confidence-acl): the Governed Memory paper (Taheri, 2026) introduces a per-fact extraction_confidence (EC) multiplier set at consolidation time and applied at recall time as a score downweight (engine.py:680–686). EC enters fused scoring as final *= clamp(ec, 0, 1) whenever RetrievalConfig.use_extraction_confidence=True (the default).

The audit risk: would EC ever consume cross-agent state, either on the write side (extractor reading another actor's memories) or the read side (recall scoring weighting Alice's candidates by Bob's EC distribution — e.g. a "calibrate against corpus mean" normaliser)?

Result: clean, no closure required. Reading the code paths:

- **Write path** (consolidation/pipeline.py:362–406): FactExtraction.run iterates over ctx.memories_created (the current run's episodes), invokes the LLM on a single episode's content, and stores the parsed `confidence` field clamped to [0, 1] on the resulting fact. No cross-agent memory is consulted; agent_id is inherited from the source episode (closed in §A.6.13).
- **Read path** (retrieval/engine.py:680–686): _final_score multiplies in only the *candidate's own* memory.extraction_confidence. There is no aggregate, no peer lookup, no normalisation against other candidates' EC distributions.
- **Storage path** (store/memory.py:294, 781):

EC is a per-row column read from the candidate's own row; no cross-row JOIN.

Pin (tests/adversarial/test_extraction_confidence_acl_side_channel.py, 7 tests, 4 parametric arms in EC-ACL-1):

- **EC-ACL-1** Alice's FACT-only ranking (content + score) is bit-identical regardless of Bob's EC distribution. Four arms: no Bob facts (control), all-1.0, all-0.0, 5-quantile spread. Pins write-side and read-side together: a future change that introduces a peer-aware EC normaliser (e.g. corpus-mean calibration) has to refresh this test.
- **EC-ACL-2** EC is a strict per-candidate multiplier: across c ∈ {0.1, 0.25, 0.5, 0.75, 0.9}, the score of a fixed Alice candidate at EC=c equals c × score(EC=1.0) to rel_tol=1e-6. Confirms the EC channel has no nonlinear / cross-candidate coupling.
- **EC-ACL-3.** With use_extraction_confidence=False, no Alice candidate's score depends on any EC value (own or Bob's). Confirms the gate has no second consumer.
- **EC-ACL-4** Positive control: with the gate ON, an Alice candidate at EC=0.05 scores < 0.5× the same candidate at EC=0.95. Pins EC-ACL-3 as a real null rather than a dead gate.

§A.6.15 closes the last open audit thread on the Governed Memory feature surface. Combined with §§A.6.6–A.6.14, the v0.2 cross-actor audit set is fully accounted for: read-side closed (§§A.6.6–A.6.11), write-side closed (§§A.6.12–A.6.14), §A.6.10 pinned as accepted behaviour, §A.6.15 audited clean.

B.11 Quorum-gated DEPRECATE (mitigation prototype for §A.6.10)

§A.6.10 pinned, as accepted behaviour, the fact that any actor that learns a `schema_id` can append CREATE+DEPRECATE and globally suppress the schema. We deferred a fix in v0.2 because every available mitigation traded against either the global-schema semantics or the debuggability of recall result rows. In this section we land the *reducer-level* prototype of a quorum gate — opt-in, defaults-off — so the production wiring can follow without further reducer churn and so the threat model now has a concrete, defended mitigation rather than just a gap.

Mechanism. SchemaLifecycleEvent gains an optional `emitter_id` field, and reduce_events gains a deprecate_quorum_k: int = 1 parameter. When `k > 1`, a DEPRECATE event no longer fires the INFERRED|PROMOTED → DEPRECATED transition on its own; instead the reducer accumulates pending_deprecate_emitters: frozenset[str] on the schema state and the transition fires only once the set has at least `k` distinct entries. Any non-DEPRECATE transition (CREATE, PROMOTE, RECOVER) clears the ballot — promotion or recovery is positive evidence the schema is alive, so partial dissent collected beforehand should not be reusable later. The default `k=1` preserves the legacy single-emitter path byte-for-byte; this is intentional, the mitigation is *available*, not *mandatory*, because the system is single-tenant by design.

Properties pinned (tests/adversarial/test_schema_deprecate_quorum.py, +14 tests):

- **Q-1, Q-9** Default `k=1` is bit-identical to the pre-quorum reducer. Q-9 is the positive control that confirms the §A.6.10 attack still works at `k=1` — its job is to fail loudly if we accidentally flip the default.
- **Q-2** Under `k=2`, a single attacker's DEPRECATE leaves the schema INFERRED with a one-element ballot. The §A.6.10 attack stops working at `k=2`.
- **Q-3** Two distinct emitters fire the transition exactly once; ballot clears.
- **Q-4** A single emitter re-voting up to k times does *not* satisfy quorum — distinctness is on `emitter_id`, not on event count. Sybil resistance proper is out of scope (the reducer trusts the emitter id), but at minimum repeated self-voting cannot bootstrap quorum.
- **Q-5** PROMOTE clears a partial DEPRECATE ballot. Otherwise an attacker who voted long ago could collude with one fresh emitter to suppress a now-promoted schema. This is the "stale dissent" bug.
- **Q-6** RECOVER (deprecated → inferred) likewise clears the ballot; the next quorum attempt restarts from zero votes.
- **Q-7, Q-8** A DEPRECATE event with emitter_id=None under `k>1` raises LifecycleViolation in strict mode and is dropped silently in non-strict mode — i.e. malformed events cannot bypass the gate by omitting the emitter.
- **Q-10** Parametric over k ∈ {2, 3, 5}: the reducer fires *exactly* at the k-th distinct vote,

never earlier and never later; intermediate ballots have the expected cardinality.
- **Q-11** Invalid `k=0` raises `ValueError` at the API boundary.
- **Q-12** The quorum fold is a pure function: same input same output across two calls.

Residual. The prototype lives at the reducer layer. The production wiring — which consolidator is allowed to attach an `emitter_id`, how `emitter_id`s are bound to actor identities, and which deployment configurations should run with `k > 1` by default — is left for the integration step. The point of pinning the prototype now is that the reducer's invariants are non-negotiable from here forward; downstream code can land without further property churn.

§A.6.16 also gives §A.6.10 a concrete defended path: multi-tenant deployments that cannot tolerate the single-actor suppression channel can opt into deprecate_quorum_k >= 2 and receive a quorum-gated lifecycle DAG with the full property suite above as the contract.

B.11.1 phase B: call-site wiring and cache identity

The phase A landing (commit `4ea3589`) threaded `emitter_id` through the wire format and the reducer; phase B (commit `71eefe9`) closed the plumbing on both ends:

- **Write side.** Config.consolidator_id: str = "" is the per-process identity of the consolidator emitting lifecycle events; it is threaded through ConsolidationPipeline.run into StageContext.consolidator_id and from there into every make_lifecycle_event(emitter_id=...) call site under SchemaUpdate. The empty default preserves byte-stable legacy emission (the `emitter_id` key is omitted from `metadata` entirely; pinned by test `B-10`). Production deployments set this to a stable per-node id (e.g. socket.gethostname()).
- **Read side.** RetrievalConfig.deprecate_quorum_k: int = 1 is plumbed through RetrievalEngine.search into CachedLifecycleSnapshot.get(deprecate_quorum_k=...), which forwards it to reduce_events. The cache treats `k` as part of the snapshot's *identity*: a stream that produced `DEPRECATED` under $k = 1$ may yield `INFERRED` (with one pending ballot) under $k = 2$ for the *same* event log, so changing k between calls forces a full rebuild. This is pinned in both directions by tests `B-6` and `B-7`; `B-9` exercises the engine→cache→reducer composition end-to-end on a real RetrievalEngine.search call.

The two knobs (Config.consolidator_id and RetrievalConfig.deprecate_quorum_k) are independent and both default to bit-identical legacy behaviour, so phase B is regression-safe for single-tenant single-node deployments while multi-tenant deployments can adopt either or both as their threat model demands.

B.12 Cross-channel coupling audit

The per-channel ACL audits (vector pool, BM25/FTS5, mechanical merge, fact extraction, write-side dedup, extraction confidence, lifecycle/cache+quorum) each pin one channel's behaviour against an adversarial peer. They are necessary but not sufficient: a side channel can hide in a *pair* of channels even when each is individually clean. We therefore catalogued every off-diagonal pair $X \times Y$ on the channel set and classified each as **I** (independent by construction — no shared cross-actor state, no shared score channel), **C+covered** (composed and pinned by an existing single test path), or **C+gap** (composed and *not* pinned by a single existing test → write a dedicated combinatorial test). The catalog and the per-pair reasoning live in research_notes/cross_channel_coupling_audit.md.

The catalog has been audited at two channel counts. At $n = 7$ (vector pool, BM25/FTS5, mechanical merge, fact extraction, write-side dedup, extraction confidence, lifecycle/cache+quorum) the surface is $\binom{7}{2} = 21$ pairs with split **11 I / 8 C+covered / 2 C+gap**. The 2026-05-24 expansion adds two read-time score channels — H (PRF entity expansion, dominance-gated) and I (share_prior reranker, rank-0-capped boost) — following the v0.2 wiring of both as runtime-toggleable knobs in RetrievalConfig. At $n = 9$ the surface grows to $\binom{9}{2} = 36$ pairs (+15 new off-diagonals); the new verdict split is **17 I / 19 C+covered / 0 C+gap**. All six read-time × write-time corners (C×H, D×H, E×H, C×I, D×I, E×I) are **I** by the read/write decoupling argument: H and I read only from the post-write actor-scoped candidate pool and have no path to the write-time channels' state. The remaining nine new pairs are all **C+covered** by the diagonal ACL audits (which

already exercise hybrid recall under PRF and share_prior) plus the §A.4.7 PRF×SP×EC panel which exhaustively pins F×I and H×I across six sweep axes at n=10 seeds with paired bootstrap CIs. The two gaps catalogued at $n = 7$ were both write-time mech-merge couplings, since closed:

- **C×E (mech-merge × write-dedup).** Adversarial scenario: Bob-write → Alice-merge interleavings could in principle let dedup-derived state from a peer leak into Alice's merge candidate set. Closed by tests/adversarial/test_cxe_mech_merge_x_write_dedup_compose.py with three deterministic two-actor invariants (CXE-1..CXE-3): under any Bob near-dup history, Alice's intra-actor merge invariant holds, Bob's rows survive composition, and the intra-Alice merge still fires.
- **C×F (mech-merge × extraction-confidence).** Adversarial scenario: could a peer's EC distribution influence which of Alice's two same-actor candidates survives a merge, or perturb the surviving EC? Closed by tests/adversarial/test_cxf_mech_merge_x_extraction_confidence.py with three invariants (CXF-1..CXF-3) sweeping Bob's EC over the four cells {low,high} × {low,high}, asserting Alice's survivor identity and the bit-identity of its EC, plus a salience-only-survivor positive control to catch any future EC-weighted refactor.

Both gap tests were green on first run. The catalog also fixes a forward decision rule for future channels: a new channel X adds n pairs, and $X \times Y$ is declared **I** with one-sentence reasoning iff X shares no per-actor state and no multiplicative/additive score contribution with Y — otherwise it is **C+gap** until a single test path exercises both. **Read/write decoupling refinement (n=9 update):** read-time score channels (F, G, H, I) cannot couple to write-time channels (C, D, E) because their input is the post-write actor-scoped candidate slice; this collapses the $X \times Y$ surface for every new read-time channel from $n - 1$ pairs to at most the prior count of read-time channels. The resulting audit-cost ceiling continues to scale linearly: at $n = 9$ the coupled subset is 19/36 (53%, all covered), and the jump from 8/21 (38%) at $n = 7$ comes entirely from H and I being read-time score channels which couple with every prior read-time channel by construction.

C Extended Related Work

This appendix expands §2's main-body summary into the full related-work coverage that an ACL 2-column 6-page Industry Track body cannot accommodate. Anchor citations + direct comparisons live in §2; the expansions below cover (1) other contemporary agent- memory designs, (2) the long-context retrieval-evaluation suite, (3) the dense-retrieval evaluation ecosystem, (4) late-interaction and learned-sparse non-inclusion, (5) pseudo-relevance feedback, and (6) the event-sourcing / bi-temporal / CRDT lineage.

C.1 Other contemporary memory designs

Beyond the three direct-comparison systems in §2.1, four further agent-memory designs warrant mention:

A-MEM (Xu et al., 2025) builds an *agentic memory* substrate as a Zettelkasten-style graph: each note is an LLM-generated atomic unit linked to others through LLM-suggested associative edges, with periodic LLM-mediated "consolidation" passes that rewrite link structure. The contrast with Engram is governance: A-MEM's link graph is rewritten whenever the consolidation LLM judges the substrate is stale, and there is no replay or audit trail across rewrites. Engram's schema lifecycle (§A7.4.4) explicitly trades off this flexibility for deterministic replay.

HippoRAG (Gutiérrez et al., 2024, 2025) routes retrieval through a knowledge graph constructed at write time and uses Personalized PageRank at query time to surface bridge entities for multi-hop questions. Where Engram's `share_prior` reranker (§A7.2) computes within-pool entity co-occurrence on the *post-retrieval* candidate set, HippoRAG operates pre-retrieval over the full graph. The two are complementary: HippoRAG addresses recall-side bridge promotion, share_prior addresses precision-side rank-0 preservation.

Cognee (cognee-ai/cognee, 2024–) is an open-source semantic memory layer combining vector retrieval with a knowledge-graph index synthesized via LLM extraction. Like A-MEM and HippoRAG, Cognee centers the graph as the substrate; like Mem0 v3, it relies on add-only writes with periodic re-consolidation. We do not benchmark against Cognee directly because it does not expose a session-level hit@k metric reproducible from raw input chats.

Zep / Graphiti (Rasmussen et al., 2025) provides a temporal knowledge-graph memory with bi-temporal edges and reified relationships, evaluated on Deep Memory Retrieval (DMR). Zep emphasises temporal coherence — what was true when — which Engram approximates through schema-lifecycle decay (§A7.4.4) but does not store as first-class temporal edges. The two designs answer different "what did the agent know" questions: Zep's is graph-shaped, Engram's is event-log-shaped.

The §2.1 "where Engram sits" framing extends to these systems: A-MEM, HippoRAG, Cognee, and Zep each center a graph substrate that is rewritten under LLM mediation. Engram's mechanical-governance bet differs categorically.

C.2 Long-context retrieval evaluation suite

The two benchmarks named in §2.2 are the agent-memory community's conversational anchors; the broader long-context retrieval- evaluation ecosystem we draw methodology from includes:

- **RULER** (Hsieh et al., 2024) — 13 synthetic tasks (NIAH variants, multi-key/value retrieval, variable tracking, frequent-words extraction, long-document QA) parametric in context length up to 1M tokens. RULER showed that nominally long-context models often degrade well below their advertised window; we adopt its **stratified-by-task** discipline at the retriever level rather than the model level, which is the methodological seed of our entity-collision per-tag stratification.
- **∞Bench** (Zhang et al., 2024) — 12 tasks averaging >100k tokens across math, code, novels, and dialogue. Used in the long-context evaluation literature as a length-stress complement to RULER's task-stress; not directly applicable to agent memory because tasks are document-centric, not session- centric.
- **LongBench-v2** (Bai et al., 2024) — 503 multiple-choice questions over 8k–2M-token contexts. Methodologically closer to multi-doc QA than agent memory.
- **NIAH / Needle-in-a-Haystack** (Kamradt, 2023) — single-fact retrieval at controlled depth. The closest one-axis ancestor of entity-collision; entity-collision generalises by stratifying on *discriminator type*, which NIAH does not.
- **LV-Eval / LooGLE / L-Eval** (An et al., 2024; Li et al., 2024; Yuan et al., 2024) — long-context QA suites that all report a single hit@k or LLM-judge accuracy per model, exhibiting the tag-mixing problem §1 motivates the entity-collision protocol to address.

C.3 Dense-retrieval evaluation ecosystem

- **BEIR** (Thakur et al., 2021) — 18-task zero-shot retrieval benchmark. Established the "BM25 is hard to beat zero-shot" finding that anchors our entity-collision protocol's BM25-floor design.
- **MTEB** (Muennighoff et al., 2023) — Massive Text Embedding Benchmark, 56 datasets across 8 task families. Source for our embedder choices: BGE-large-en-v1.5 sat near the top of MTEB's retrieval leaderboard at the time of our encoder grid freeze, which is why we extended the protocol to it as the encoder-capacity falsification test.
- **MS MARCO** (Bajaj et al., 2016) and **TREC Deep Learning** (Craswell et al., 2020, 2021) — passage-retrieval staples. Useful as a sanity prior for relative encoder ordering but tangential to the agent-memory setting because queries are short and corpora are static.

C.4 Late-interaction and learned-sparse baselines

The v0.2 measurement grid covers three **single-vector dense encoders** (HashTrigram-256, MiniLM-384, BGE-large-1024). We deliberately do not include late-interaction (ColBERT, ColBERTv2; Khattab and Zaharia, 2020; Santhanam et al., 2022) or learned-sparse retrievers (SPLADE, SPLADE++; Formal et al., 2021, 2022) — both are documented to outperform single-vector dense encoders on BEIR — because the headline question this paper engages is specifically **whether per-query semantic capacity in the single-vector regime is the binding constraint on agent-memory retrieval**. The two-axis result of §4.3 and the encoder-capacity falsification of §A.4.16 are claims about that regime: a 2.7×-parameter increase within the single-vector family does not collapse the lexical-vs-intent split, which is the methodological finding that motivates the protocol. ColBERT and SPLADE answer a different question — whether *interaction structure* (token-level late interaction) or *index sparsity*

(learned sparse projections) recovers cells the single-vector family cannot — and a clean answer to that question requires its own protocol design, not a 4th column on this grid. We accordingly flag late-interaction and learned-sparse retrievers as a **v0.3 follow-up** with its own protocol freeze, not a v0.2 omission.

A secondary, deployment-side consideration reinforces this scope boundary: ColBERT's per-token storage and SPLADE's per-query inference cost both invert the deployment trade-off table in §3.1.1 on commodity-CPU hosts (the v0.2 testbed and the v0.2 default-embedder choice). A reviewer interested in the accelerator-only deployment regime should read §A.4.16 alongside this section: the same trade-off applies to BGE-large there, and ColBERT/SPLADE land further along the same axis.

C.5 Pseudo-relevance feedback

- **RM3** (Lavrenko and Croft, 2001) — the canonical relevance-model PRF expansion that mixes a discriminative term distribution via learned λ. §5.1 details why our heuristic-PRF falsification (§A.4.15j-o) does not extrapolate to RM3. AUDIT-D ships an RM3 arm across the entity- collision grid, BEIR FiQA, and LongMemEval n=500 (§A.4.16.4); the headline finding is that RM3 does not rescue PRF on intent-style queries, sharpening §4.3's two-axis claim.

- **Rocchio relevance feedback** (Rocchio, 1971) — original vector- space PRF. Cited for completeness; supplanted by RM3 in modern retrieval-evaluation practice.

C.6 Schema-lifecycle and event-sourced memory

The schema-lifecycle invariant set we discuss in §A7.4.4 and §A4.2 draws on three threads:

- **Event sourcing** (Fowler, 2005; Vernon, 2013) — pattern from domain-driven design where state is the fold of an immutable event log rather than a mutable record. Our schema lifecycle is an event-sourced reducer (tests/property/test_schema_lifecycle.py).

- **Bi-temporal data modelling** (Snodgrass, 1999; Date et al., 2002) — the discipline of separating "what was true" from "when we knew it was true." Engram approximates the latter through write timestamps on the decision log; we do not claim full bi-temporal correctness.

- **CRDT / monotone reducer literature** (Shapiro et al., 2011) — informs our "decay is monotone in real time" invariant (test_schema_decay.py).

D Extended Discussion

This appendix expands §5.1's main-body operational rule with the four supporting analyses that the ACL 2-column 6-page Industry Track body cannot accommodate. Each is referenced by a one-sentence pointer in §5.1; the full content lives here.

D.1 Why does adaptive vector-weight routing fail?

The 11.7pp oracle gap on LoCoMo is real — there exists a per-query optimal vw, and switching at query time would close the gap. But the gap/crowdedness signals we tested do **not** localize that optimal. Two hypotheses:

1. **Signal coarseness.** bm25_top1 - bm25_top2 only sees the top-2 distance; it misses the broader candidate distribution.
2. **Confounding with hardness.** A small gap may indicate "BM25 is uncertain" (good signal) **or** "all candidates are semantically near the gold" (bad signal — vector won't help either). The signals collapse both regimes.

We additionally trained a GradientBoostingClassifier over the full BM25 feature panel + category one-hot under leave-one-conversation-out CV across all 10 LoCoMo samples (§4.4, evals/locomo_learned_router.py). The learned router is also null (Δhit@1 = -0.0005 [-0.0030, $+0.0015$] on HT; ST identical within Monte-Carlo noise). The router degenerates to vw=0 because 1801/1978 queries already prefer vw=0 in the oracle. The 11.7 pp headroom is real but unrecoverable from any pre-routing signal we can observe; gold position is required, which by definition cannot exist at decision time. We accordingly *reframe* vector fusion as a **paraphrase-robustness** mechanism rather than a per-query precision lever, and validate that framing on LongMemEval (§A.4.8.2).

Verdict. Adaptive vw routing is a measured null on LoCoMo. The correct operational role for vector fusion is paraphrase-robustness, not per-query precision routing.

D.2 Schema lifecycle as a research artifact

Why this section exists. A reviewer may reasonably ask why a paper about retrieval-evaluation methodology spends discussion on schema-lifecycle invariants. The answer is the bridge to §1's "deterministic governance is a prerequisite, not a contribution": the cross-encoder paired-bootstrap inversions in §A.4.16.3 (n=100 → n=500) only make sense if re-running the same ingest against the same dataset produces the same memory store byte-for- byte. The three properties below are what give us that guarantee. They are *machinery for the methodology*, not a retrieval claim — §A.4.6 falsified the retrieval interpretation directly, and §A4.3 below restates the consolidation claim accordingly.

§A.4.6's bisection landed the operational claim that the lifecycle is not a retrieval mechanism. What it *is* a mechanism for is **deterministic governance under replay**. We formalize three invariants on the lifecycle reducer, each verified by exhaustive execution traces over the property-based testing substrate; the specific test file paths and per-property case counts are catalogued in the implementation traceability index (§A6).

1. **Lifecycle decisions are events, not in-place mutations.** A schema's state at time t is the fold of its decision log up to t. This is the same discipline event-sourced ledgers borrow from accounting; in a memory system it gives bit-identical audit replay across re-runs of the same ingest stream.
2. **Family clustering is decision-stable under permutation.** For any permutation of the input fact stream, the *family assignment* a fact lands in is invariant; only the order of decisions within a family is permuted. This is what makes the lifecycle safe to run concurrently: writers don't race for cluster identity.
3. **Decay is monotone in real time, not in arrival time.** A fact's confidence trajectory depends only on wall-clock spacing, not on whether other facts were observed between ticks; the property is verified under fuzzed interleaving of `tick()` and `update()` calls. This rules out a class of write-amplification bugs where a chatty witness inadvertently extends an unrelated fact's half-life.

These properties generalize beyond Engram: any memory system that wants deterministic replay — audit trails, regression-debug reproduction, cross-machine rehydration — needs all three. The lifecycle's value is not retrieval lift but the substrate it provides for every other claim in the paper: §A.4.7's PRF×SP operating point is only defensible because the ingest state is replayable from the decision log.

D.2.1 Lifecycled schemas vs Letta-style memory blocks

Letta's `human`/`persona` blocks (Packer et al., 2023) and Engram's SCHEMA memories are adjacent in design space — both are named, mutable agent state — but differ on three axes that matter for governed deployment.

1. **Mutation discipline.** A Letta block is a string the LLM rewrites in place via tool calls; prior content is reachable only through external chat history. An Engram schema is a fold over an append-only decision log (§A7.4.4): every state change is a typed event with a reason field, and previous state is recoverable by replaying any prefix. Letta optimises for prompt-window compactness; Engram for audit-grade replay.
2. **Identity stability under adversary.** A Letta block is identified by name; whoever can issue a tool call can rewrite it. Engram schemas are content-addressed (cluster centroid + family key, §A7.4.4) with a quorum-gated DEPRECATE primitive (§A.4.6, §A.6.16) requiring k independent emitters over a w-event window. A single compromised emitter cannot take a schema down.
3. **Recovery semantics.** Letta has no first-class undo: a corrupted block must be reconstructed from chat history by the same LLM that may be the corruption source. Engram's lifecycle DAG includes a RECOVER edge — verified under randomized event interleaving (see §A6) — that re-promotes a DEPRECATED schema once subsequent evidence reaches the same quorum. RECOVER is path-dependent on the decision log, not on current LLM judgment.

The trade is real: Letta is cheaper to operate (no append-only log, no quorum bookkeeping) and for

chat-assistant workloads the savings dominate. Engram's bet is that the substrate cost is recovered the first time a regulator, debugger, or post-incident reviewer asks "what did this agent know and when" — a question Letta's named-block design cannot answer without external instrumentation.

D.3 The honest version of "consolidation lifts retrieval"

§A.4.6 forces a re-statement of the consolidation claim. A naive reading of the §87 pipeline says "more stages, more lift." The bisection says otherwise: on the Mem0-shaped LoCoMo10 fixture, **one stage (episode extraction) carries the full retrieval delta, and seven of the eleven downstream stages emit identically-zero per-pair diffs against any prefix that already includes extraction.** Two of the four non-trivially-moving downstream stages (schema_update, `appraisal`) are *negative* on Δhit@1 at point-estimate, and neither survives a paired bootstrap.

This is stronger than "the rest of the pipeline doesn't help": it falsifies stage-additive lift attribution. Stage ablations that report a single end-to-end metric and treat a positive delta as evidence-of-mechanism are under-specified — the lift was already present before the ablated stage ran. Our v0.2 minimum: **leave-one-out necessity + paired bootstrap on the per-question diff**, with multiple-comparison cost paid explicitly.

The architectural implication: the lifecycle stages are **not retrieval mechanisms**. They are governance — dedup at write, schema as a writable cluster, appraisal as a salience signal for downstream policy. They earn their place on §A4.2's grounds, not §4's.

Verdict. Lifecycle consolidation does not improve retrieval; it provides governance. Future ablations of "consolidation" features must report leave-one-out + paired-bootstrap with multiple-comparison cost paid explicitly.

D.4 The PRF latency myth

A controlled cProfile re-measurement (§A.4.15-profile, n=30 k, 200 paired queries) falsifies the staged claim that PRF "doubles recall p50." PRF-only p50 is *0.86 ms below* baseline (40.08 vs 40.94) and PRF-only p95 is *4.7 ms below*; share_prior-only matches baseline within noise. Only the combined PRF×share_prior arm shows real overhead (+14 % p50, +24 % p95). The dominant cost across all arms is sqlite3.Connection.execute (~73 % of cumulative recall time); PRF doubles engine.search call count but adds only ~6 % to its cumulative cost because the second pass hits warm pages.

Generalizing: single-shot microbenches with small n on a hot data path are noise-dominated at the ms scale. Latency claims for a retrieval lever must be paired (same query stream, warm cache) and reported as a percentile distribution, not a mean. v0.2 standing rule: no latency claim ships without $n \geq 200$ paired queries and matched ingest. The candidate-pool prune for v0.3 is justified for the `both` arm only.

Verdict. Single-shot ms-scale microbenches are noise-dominated. Standing rule: paired $n \geq 200$ + percentile distribution, or no latency claim.

E Extended Threats

This appendix expands §6's main-body threats with the supporting analyses that the ACL 2-column 6-page Industry Track body cannot accommodate. Each is referenced by a one-sentence pointer at the end of §6; the full content lives here.

E.1 Sibling-lexical paraphrase replication on `service`

§6.1 reports the paraphrase replication on the strongest lexical cell (`tool`). To check whether the paraphrase collapse is `tool`-specific or generalizes across the lexical axis, we re-ran the same protocol on `service` (closed-vocabulary proper-noun answers: aws, gcp, azure, …):

Compared to the fixed-template baseline (hash service K=16: +0.057 [+0.025, +0.088]), the paraphrased lift shrinks to +0.037 — about 65% of the fixed-template effect — **but stays CI-strictly above zero**. Unlike `tool`, hash on `service` survives memory paraphrase at K=16. The conservative reading: the paraphrase collapse on `tool` is real, but the broader claim is not "hash trigrams die under paraphrase" — it is "hash retention under paraphrase depends on how lexically distinctive the answer vocabulary is at the character-trigram level." `service`'s answer set (short proper nouns: aws, gcp, azure) preserves more discriminative character-trigrams across template variation than `tool`'s (git, docker, postgres). The two-axis claim survives on the **lexical/intent split**, but the within-lexical paraphrase robustness has tag-level structure.

Outputs: bench/results/ec_sweep_{hash,st}_

K	hash Δhit@1 (paraphrased)	ST Δhit@1 (paraphrased)
2	−0.094 [−0.219, +0.031]	+0.109 [+0.047, +0.188]
4	−0.031 [−0.102, +0.039]	+0.156 [+0.094, +0.219]
8	+0.031 [−0.008, +0.070] **null**	+0.141 [+0.098, +0.188]
16	**+0.037 [+0.012, +0.062]**	**+0.105 [+0.078, +0.133]**

service_n32_K16_paraphrased{,_ci}.json.

E.2 Single embedder per family

§6 tests exactly one hash-trigram dim (256) in the headline figure and one sentence-transformer (MiniLM-L6-v2). To check whether the lexical-axis hash lift is specific to dim=256, we ran a hash-dim ablation at the strongest lexical cell (`tool`, K=8, n=32):

Dim=128 is below noise; 256–1024 sit on a CI-positive plateau with no monotone scaling. The two-axis claim is robust across hash dim $\in$ {256, 512, 1024}; only the smallest sketch (128) collapses. Extending the embedder grid to BGE / E5 on the dense side and to character-quintgrams on the sketch side would tighten the family generalization claim. The BGE-large-en-v1.5 (1024-d) follow-up has since landed (Appendix §A.4.16) and rejects the encoder-capacity hypothesis: the two-axis result survives a 2.7×-parameter encoder swap, with BGE *losing* on lexical-discriminator tags. The character-quintgram sketch is left as future work and flagged as a named scope limitation rather than an open TODO.

E.3 hit@1 only

§6 reports `hit@1` because it is the worst-case metric and the most sensitive to retriever ranking. `hit@5` and MRR mostly converge toward 1.0 on the entity-collision corpus and are uninformative. LoCoMo numbers are reported across all metrics in SCALE_REPORT.md.

E.4 Single-process SQLite

All operational latency numbers are single-writer SQLite/FTS5. Multi-process write contention is not measured. A concurrency torture suite ($\geq$50 writers × $\geq$50 readers, see §A6) passes correctness invariants under contention, but throughput-under- contention is not yet a reported number.

E.5 Single machine, single OS

All wall-clock and throughput numbers come from one Linux x86_64 workstation (see REPRODUCIBILITY.md for the full env). p50/p95/p99 ingest latencies and the 100k constant-p99 claim therefore should be read as *invariant within this hardware envelope*. The replay discipline argued for in §A4.2 is what makes cross-machine reproduction tractable — point-estimates may shift, but the event-sourced lifecycle guarantees that the *shape* of any reported distribution is reproducible from a pinned decision log; the diff-results acceptance gate (REPRODUCIBILITY.md §4) operationalizes this with ±0.5pp / ±25%-latency tolerances.

E.6 Author-as-annotator on tag definitions

The lexical/intent dichotomy of §3.3 is author-defined: `service` and `tool` were labeled as closed-vocabulary lexical, the other three as open-vocabulary intent-style, without an inter-annotator agreement protocol. The labels are derived from the answer-set construction (closed enum vs free phrasal slot), not from a third party's judgment, so the categorization is *traceable* but not *independently validated*. A reviewer could plausibly relabel `technical` as lexical and recover a different two-axis fit. We therefore present the dichotomy as a hypothesis the data is consistent with, not as the unique correct partition.

F A6 Implementation and Testbed Traceability

This appendix is a one-stop reference for the implementation paths and reproduce scripts cited by abstraction in the body. Reviewers who do not need to inspect the testbed code can skip it; reviewers who *do* — particularly when checking the deterministic-replay arguments in §A7.4, §A4.2, and §75 Limitations — will find every cited file here, with what it pins and how many cases / properties it exercises.

We treat this index as version-controlled supplementary material. Every path below is verified to exist at the tagged release by scripts/verify_repro_artifacts.sh, the same script that gates the artifact registry (§REPRODUCIBILITY).

dim	Δhit@1	95% CI
128	−0.0195	[−0.0742, +0.0312]
256	+0.0664	[+0.0039, +0.1250]
512	+0.0586	[+0.0000, +0.1172]
1024	+0.0703	[+0.0117, +0.1250]

F.1 Pure-reducer invariants for the schema lifecycle (§A4.2)

The three lifecycle invariants in §A4.2 — event-sourced state, permutation-stable family clustering, real-time monotone decay — correspond to the following property-test files:

Hypothesis is configured at max_examples=200 per property in CI; the lifecycle reducer is therefore exercised over ≈1.6k randomized event traces per CI run.

F.2 Write-path independence (§A7.4.2)

The two invariants on deduplication × extraction-confidence independence (I1/I2 in §A7.4.2) are pinned by:

F.3 Concurrency torture suite (§75 Limitations)

The ≥50 writers × ≥50 readers correctness suite referenced as testbed sanity in §75 Limitations lives at:

The suite asserts correctness invariants (no torn writes, no lost events, ACL never lifts under contention) but does not yet report throughput; this is the §75 Limitations limitation.

F.4 Adversarial / ACL invariants (Appendix A2)

The path-by-path enumeration of the 11 ACL side-channel audits is already in §A2 (Security Audits). §A6 is intended for body-level abstractions; readers tracing §A2 references should consult that appendix directly. The full list of tests/adversarial/ test_*.py audit files is duplicated below for convenience:

F.5 Scale evidence (§A.4.14, §6.4)

The 1M-memory single-writer characterization referenced in §1, §A.4.14, and §6.4 corresponds to:

F.6 Why this index lives here, not in the body

Three of Engram's invariants — event-sourced lifecycle, mechanical merge, write dedup — are implementation-level guarantees that support the measurement claims rather than being measurement claims in their own right. The body of the paper therefore states each invariant in academic abstraction (deterministic fold over events, permutation-stable cluster identity, real-time monotone decay) and defers per-property file paths and case counts to this appendix. This separation matches the standard distinction in systems papers between *what is true of the system* (body) and *how the truth is verified at the implementation* (appendix), and it leaves the body sections short enough that a reviewer focused on the entity-collision protocol can read the methodology subsections in linear time.

G Extended Methods

This appendix expands §3's main-body protocol description with the supporting method detail that the ACL 2-column 6-page Industry Track body cannot accommodate. Each subsection is referenced by a one-sentence pointer at the end of §3.3; the full content lives here.

G.1 LoCoMo and adaptive-vw experiment

For the adaptive-vw null result, we use the LoCoMo adapter (evals/locomo_adapter.py) over n=1978 questions and compute (a) the per-query oracle hit@1 (best vw per query), (b) static-best vw, and (c) tau-thresholded policies on bm25_top1 - bm25_top2, normalized gap, and crowdedness@0.95.

G.2 share_prior reranker (Personize §96 adaptation)

We adapt the *shared prior* signal from Personize (§96) into a candidate-pool reranker. Rather than scoring each candidate against the query alone, we build an undirected entity-sharing graph over the top-`pool_size` fused candidates: an edge connects two candidates whenever their extracted named-entity sets intersect. Each candidate's *multi-mate degree* — the number of pool members it shares at least one entity with — is a within-pool

Invariant (§A4.2)	Test file (tests/property/)	Notes
Lifecycle state = fold of decision log	test_schema_lifecycle.py, test_schema_decision.py, test_schema_decision_x_reducer.py	Includes RECOVER edge fuzzing
Family clustering invariant under permutation	test_schema_family.py, test_schema_family_decision.py, test_schema_family_window.py	Permutes the input fact stream; asserts assignment
Decay monotone in real time, not arrival	test_schema_decay.py	Fuzzes interleaved `tick()` / `update()` calls

Invariant (§A7.4.2)	Test file
I1: dedup outcome invariant under keeper-side confidence	tests/property/test_dedup_extraction_confidence_indep.py
I2: deduped write does not mutate keeper's confidence	(same)

popularity signal that, on multi-hop bridge queries, fires for the bridge fact even when the bridge fact does not lexically match the query string. We add a bounded boost $\alpha \cdot$ deg / max_deg to each candidate's fused score and re-sort.

Rank-0 preservation invariant. We cap the per-candidate boost so that no non-rank-0 candidate's post-boost score can equal or exceed the original rank-0 score: $\text{score}'_i = \text{score}_i + \min(\alpha \cdot \text{deg}_i/\text{max\_deg}, \text{score}_0 - \text{score}_i - \varepsilon)$.

By construction $\text{score}'_i < \text{score}_0$ for all $i \geq 1$, so `hit@1` is mathematically — and, as we verify in §4, empirically — never regressed. The reranker is therefore safe to ship default-off behind a flag and accept-only at higher k. Verified across 5 seeds $\times$ 5 α values $\times$ 2 recipes $\times$ 3 pool sizes = 150 arms: Δhit@1 $\equiv$ 0.000 everywhere (SHARE_PRIOR_REPORT.md, all tables).

Protocol. We evaluate on two synthetic recipes that probe orthogonal failure modes: **Unique-entity** (do-no-harm; each gold has a distinct anchor entity, no graph structure; share_prior expected inert at hit@1 and at most mildly noisy at hit@k); and **Bridge** (target signal; each query is a pair of facts joined by a shared anchor entity; gold pair facts share a degree-2 hub in the graph and should be promoted).

For each recipe we sweep $\alpha \in$ [0.02, 0.30], pool_size $\in$ {20, 40, 80}, and entity_weight $\in$ {0.0, 0.1, 0.2, 0.3} (the channel that also lives in the fused scorer), across seeds `{42..46}` and corpus scales from 60 to 120 query pairs with up to 200 distractor facts. We report paired Δ`hit@k` and Δpair_recall@k versus the same fused-scorer baseline (no rerank) on identical seeds, with n=10 paired-bootstrap 95% CIs (§A.4.7).

Pool-size dilution. As `pool_size` grows, the deg / max_deg distribution becomes less peaked; the relative ordering between the bridge fact and the rest of the pool flattens. We confirm this empirically (pool_size $\in$ {20, 40, 80} at α = `0.10`, `ew = 0.10` on the corrected small bridge corpus: Δpair@10 = {+0.050, $-$0.025, +0.025}; §A.4.7).

Adaptive α (opt-in regularizer). A constant α boosts identically whether the candidate pool's most-shared entity has degree 1 or degree 9, even though the rank-0 cap already saturates the "graph is a near-matching" case. We schedule a tapered multiplier on `max_deg`, $\alpha_{\text{eff}}(\text{max\_deg}) = \alpha \cdot 1/(1+\max(0, \text{max\_deg} - 1)/4)$,

i.e. $\alpha_{\text{eff}} = \alpha$ at max_deg $\in$ {0, 1} and decays monotonically toward 0 as the pool densifies (max_deg = 5 $\rightarrow$ 0.5α, max_deg = 9 $\rightarrow$ α/3). This lives behind RetrievalConfig.share_prior_adaptive_alpha (default `False`) and ships as a hedge against α over-shoot rather than an unconditional improvement; §A.4.7 reports the empirical regime where it pays off.

G.3 PRF entity expansion (dominance-gated)

Reranking cannot promote what the first-pass top-K never admitted, so we close the upstream gap with pseudo-relevance feedback (PRF) restricted to entity tokens. Given a first-pass top-K, we extract entities from each of the top-top_k_for_prf results and rank them by document frequency within the pool. We then expand the query with the most-dominant entity *only if* it appears in at least min_dominance $\cdot$ top_k_for_prf of the first-pass results; otherwise we issue the original query unchanged. The dominance gate is the do-no-harm guarantee: when no single entity dominates the first-pass pool (the typical unique-entity workload), we do not expand at all, so unique-recipe `hit@1` is not regressed by classic PRF over-expansion.

The expansion is wired into RetrievalEngine. search behind RetrievalConfig.query_expansion_min_dominance: float | None (default `None` = off). At runtime, when a non-`None` value is configured, the engine performs a first-pass retrieval, computes the dominance-gated entity, and (if the gate fires) re-issues a single expanded query whose results replace the first-pass results for the user-visible top-k. The operating point we recommend, defended in §A.4.7, is min_dominance = 0.3 with top_k_for_prf = 20.

Concern	Test file (tests/concurrency/)
Generic write/read race coverage	test_race_conditions.py
Dedup race	test_dedup_race.py
High-fanout writer/reader load	test_high_fanout.py
JSONL append-buffer concurrency	test_jsonl_buffer_concurrency.py
JSONL truncate race	test_jsonl_truncate_race.py

Audit subject (§A2)	Test file (tests/adversarial/)
PRF expansion ACL	test_prf_acl_side_channel.py
PRF + IDF gate ACL	test_prf_idf_acl_side_channel.py
Share-prior ACL	test_share_prior_acl_side_channel.py
Lifecycle cache ACL	test_lifecycle_cache_acl_side_channel.py
Schema-id targeted suppression	test_schema_id_targeted_suppression.py
Vector-channel ACL	test_vector_channel_acl_side_channel.py
Mechanical-merge ACL	test_mechanical_merge_acl_side_channel.py
Fact-extraction ACL inheritance	test_fact_extraction_acl_inheritance.py
Write-dedup ACL	test_write_dedup_acl_side_channel.py
Write-dedup × ACL race	test_write_dedup_acl_race.py
Extraction-confidence ACL	test_extraction_confidence_acl_side_channel.py
Schema deprecate quorum	test_schema_deprecate_quorum.py
Mech-merge × write-dedup composition (CXE)	test_cxe_mech_merge_x_write_dedup_compose.py
Mech-merge × extraction-confidence (CXF)	test_cxf_mech_merge_x_extraction_confidence.py

G.4 Governed Memory integration (secondary systems note)

Scope of this section. §A7.4 documents the four write-path primitives (dual extraction, cosine dedup, mechanical merge, schema lifecycle) that make our paired-bootstrap CIs replayable. Reviewers focused exclusively on the entity-collision protocol may skip this subsection: nothing in §4 depends on schema- lifecycle behaviour beyond the testbed-determinism requirement we surface here. The lifecycle's *own* retrieval behaviour (and its non-contribution to recall) is the subject of §A.4.6's bisection and §A4.2's discussion — also orthogonal to the headline two-axis claim.

Engram's write/consolidation path adopts four primitives from the Personize Governed Memory proposal (Taheri, 2026). This subsystem is a v0.2 systems contribution; the paper's headline claims do not depend on it.

G.4.1 Dual extraction with per-fact confidence

FactExtraction returns each fact with extraction_confidence $\in$ [0, 1]; RetrievalConfig.use_extraction_confidence (default `True`) multiplies the fused score by this factor and records it in ScoredMemory.sources.

G.4.2 Write-side cosine dedup at 0.92

StorageConfig.write_dedup_threshold = 0.92 gates store.upsert against the in-process vector index. This is a pure write-path filter — not a merge — orthogonal to extraction confidence. The independence is formalized as two invariants (I1/I2): the deduplication outcome is invariant under keeper-side confidence, and a deduped write does not mutate the keeper's confidence. Both invariants are verified by randomized property tests over the joint state space; case counts and traceability in §A6.

G.4.3 Mechanical merge (no-LLM)

MechanicalMerge (Stage 12) walks store.all_active() and, for any pair above merge_threshold = 0.95, suppresses the lower- salience member. No LLM call; order-stable; idempotent under fixed embeddings.

Claim	Test file (tests/scale/)
Tail-100k p99 write latency at 100k	test_ingest_scale.py::test_scale_recall_after_100k
Same-harness 1M extension	test_ingest_scale.py (1M fixture)

G.4.4 Schema lifecycle as a pure reducer

schema_lifecycle.py folds an immutable SchemaLifecycleEvent log into a {schema_id: SchemaState } snapshot. The transition DAG is INFERRED → PROMOTED → DEPRECATED, INFERRED → DEPRECATED, DEPRECATED → INFERRED (recovery only, fresh `window_id`). Five invariants are enforced: transition closure, CREATE-once, no-op on unknown id, version monotone under status changes, and recovery freshness. RetrievalConfig.respect_schema_lifecycle (default `True`) filters DEPRECATED candidates before scoring. Each invariant's Hypothesis property test and the bug-class it catches are consolidated in §A.4.17.

G.4.5 What we did not adopt

We omit Personize's LLM-mediated merge (replaced by §A7.4.3's mechanical merge) and their governance-review stage (no analogue in single-user deployments).

G.5 Discriminator tags — full schema

Tag selection follows the construction rule (closed enum vs free phrasal slot, §3.3, §75 Limitations Author-as-annotator). The vocabulary column is what determines the lexical/intent split that the two-axis result of §4.3 isolates.

tag	example query	example answer set	vocabulary
`preference`	“what does Alice prefer for X?”	dark mode, light mode, …	open phrasal
`project`	“what is Alice working on?”	varied open phrases	open phrasal
`technical`	“what does Alice use for X?”	varied open phrases	open phrasal
`service`	“what service does Alice use?”	aws, gcp, azure, …	closed lexical
`tool`	“what tool does Alice use?”	git, docker, postgres, …	closed lexical